

ECE 5760 Lab 2: Mandelbrot Set Visualizer

Md Shad, Michael Grillo, Stephen Barnett

March 6, 2026

1 Introduction

In this lab, we designed a Mandelbrot set visualizer on the DE1-SoC FPGA where we computed and rendered a high-resolution fractal in real time. The Mandelbrot set is generated by repeatedly calculating the complex iteration $z_{n+1} = z_n^2 + c$ using an initial condition of $z_0 = 0$, where each complex constant c corresponds to a pixel on the screen. For every pixel in the 640×480 display, the system determines whether the sequence diverges by evaluating whether the magnitude of z exceeds a threshold value. The color of each pixel is determined by the number of iterations required for the sequence to diverge. This produces the familiar fractal boundary and allows regions of stability and instability in the complex plane to be visualized.

The core computation was implemented in synthesizable Verilog and executed entirely within FPGA hardware using fixed-point arithmetic to efficiently perform the repeated multiplications and additions required for each Mandelbrot iteration. In order to accelerate the rendering, we designed a parallel architecture containing multiple iterator pipelines that operated simultaneously. Each of these iterators computed the iteration count for a smaller subset of the screen's pixels and stored the resulting color values inside dedicated on-chip M10K memory blocks. This organization of memory reduced contention between solvers and allowed the system to compute thousands of pixels concurrently.

The Hard Processor System (HPS) provided high-level control and enabled user interaction by communicating with the FPGA logic through memory-mapped parallel I/O (PIO) interfaces configured in Platform Designer. Through these registers, the HPS specifies the complex plane region that should be rendered, including the starting and ending real and imaginary coordinates as well as the step sizes between adjacent pixels. These parameters define the viewport of the Mandelbrot set and allow the user to reposition or zoom into specific regions of the image. Whenever the HPS signals that the viewport has been adjusted, the FPGA recomputes the fractal for the new coordinate range. A primary objective of this lab was to utilize as many FPGA hardware resources as possible, particularly DSP blocks and on-chip memory, in order to maximize parallelism and achieve the fastest possible rendering acceleration.

The computed pixel values are written into iterator-specific memory blocks, and the VGA controller reads and multiplexes the outputs of these memories to drive the 640×480 display. The system therefore renders the Mandelbrot set directly using FPGA-generated data without routing pixel values through the HPS. The display also supports interactive

exploration of the fractal, allowing users to pan and zoom into detailed regions of the set while observing the complex boundary structures that emerge. Users interact with the system through a mouse interface, which allows them to move a cursor across the display and select regions of the image to zoom into or pan across the complex plane. Through careful hardware parallelization, fixed-point design, and hardware–software co-design, our implementation demonstrated fast fractal rendering, efficient use of FPGA resources, and responsive user interaction through the HPS.

2 Mathematical Background

The Mandelbrot is defined as the parameter space where the equation through the complex space where $z_{n+1} = z_n^2 + c$ is bounded given an initial condition, $z_0 = 0$. For each value of $c \in C$, the complex sequence is iterated for a set number of iterations (usually 1000 iterations), and checked for divergence.

Typically, the Mandelbrot set is calculated over a small region $\in C$, where the fractal appears. This region is more precisely, values $c_r \in [-2, 1]$ and $c_i \in [-1.5, 1.5]$. To represent these points in the equation $z_{n+1} = z_n^2 + c$, c is defined as $c = c_r + ic_i$, where c_r is the real component of c and c_i is the imaginary component of c .

The Mandelbrot solver iterates for each value within the rectangular region of the complex plane with $c_r \in [-2, 1]$ and $c_i \in [-1.5, 1.5]$ and for each value, runs the iterative function until either the system "doesn't diverge" meaning that the system incurs 1000 iterations without exceeding the predefined bounds of divergence, which is mathematically determined to occur if $|z| > 2$.

Mathematically, the divergence is determined by evaluating the magnitude of the complex value $z = z_r + iz_i$, which is defined as $|z| = \sqrt{(z_r)^2 + (z_i)^2}$, or in addition, $|z|^2 = (z_r)^2 + (z_i)^2$. It has been previously proven that if $|z| > 2$, the sequence will diverge to infinity. Therefore, our solver checks if $|z|^2 > 4$ or $4 < (z_r)^2 + (z_i)^2$, and if so, this is the iterated where we know that the sequence has diverged.

To solve for the "Mandelbrot set visualization", each value of this rectangular region of the complex plane is calculated, and the output of each iterator, which is the iteration value where the outcome of the system is determined, whether that is the iterator value where we determine the system diverges, or 1000 if the system does not diverge is returned. To get the colorful images of the Mandelbrot that is commonly viewed online, each of these iterator values is mapped logarithmically to a specific color, with the value representing 1000, or convergence of the set, as the color black, and different colors representing difference speeds of divergence.

3 Roles of the ARM (HPS) and FPGA

A hardware–software co-design approach was used to implement the Mandelbrot visualization system, with the ARM-based Hard Processor System (HPS) handling system control and user interaction while the FPGA handled the computationally demanding fractal calculations. The HPS managed configuration parameters, user inputs, and communication with

the hardware accelerator, while the FPGA executed the highly parallel Mandelbrot iteration pipelines. This separation allowed specialized hardware to handle the computational workload while maintaining a flexible and interactive software interface.

3.1 FPGA Responsibilities

The FPGA implemented the core Mandelbrot computation engine. Multiple iterator pipelines were instantiated in hardware in order to evaluate many pixels simultaneously. Each iterator repeatedly computed the Mandelbrot iteration equation using fixed-point arithmetic, which allowed the arithmetic operations to efficiently map onto the Cyclone V DSP blocks and logic resources. These pipelines performed the multiplications, additions, and comparisons required to determine whether each complex value diverged and how many iterations were required before divergence occurred.

By executing these computations entirely in hardware, the FPGA provided deterministic execution and high throughput. The parallel iterator pipelines enabled the system to evaluate many pixels concurrently, making the rendering of the Mandelbrot set significantly faster than a sequential software implementation. Each iterator wrote the computed pixel color values into on-chip memory blocks, which were later read by the VGA subsystem to display the fractal image.

The FPGA also exposed a set of memory-mapped control registers that served as the interface between the HPS and the hardware accelerator. These registers stored parameters that defined the region of the complex plane being rendered, including the starting and ending real and imaginary coordinates, as well as the step sizes used between adjacent pixels. The FPGA continuously read these registers and used their values to determine the coordinates that each iterator pipeline should evaluate.

3.2 ARM (HPS) Responsibilities

The ARM processor running on the HPS acted as the high-level controller of the system. Rather than performing the Mandelbrot calculations directly, the processor was responsible for configuring the FPGA accelerator and managing user interaction. Through memory-mapped I/O, the HPS wrote the parameters that defined the viewing window of the Mandelbrot set, including the start and end coordinates and the step sizes for the real and imaginary axes.

The HPS also managed the interactive user interface. A C program running on the processor monitored keyboard input and adjusted the fractal viewing region accordingly. By modifying the coordinate ranges, users could pan across the complex plane or zoom into specific regions of the Mandelbrot set. Once these parameters were updated, the HPS wrote the new values to the FPGA registers, causing the hardware accelerator to recompute the fractal for the updated region.

Another responsibility of the HPS was monitoring system performance. The FPGA recorded the number of clock cycles required to compute a frame and stored this value in a hardware register. The HPS read this value, converted it into milliseconds using the FPGA clock frequency, and displayed the rendering time to the user. This allowed users to observe how quickly the hardware accelerator rendered the Mandelbrot set.

3.3 System-Level Integration

Separating computation and control between the FPGA and the HPS resulted in an efficient system architecture. The FPGA operated as a high-speed numerical engine dedicated to Mandelbrot calculations, while the HPS provided flexibility for configuration, interaction, and monitoring. Communication between the two components occurred through memory-mapped registers, which simplified debugging and system integration.

4 RTL Design

This design implements a hardware accelerator for generating a Mandelbrot set visualizer, using 4.23 signed fixed point arithmetic on an Altera DE1-SoC. This system primarily calculates the iterative Mandelbrot generation equation: $z_{n+1} = z_n^2 + c$, for values $c_r \in [-2, 1]$ and $c_i \in [-1.5, 1.5]$, where $c = c_r + ic_i$. The components of z can be expanded into $z_{r,n+1} = c_r + (z_{r,n}^2 - z_{i,n}^2)$ and $z_{i,n+1} = c_i + 2z_{r,n}z_{i,n}$. Each clock cycle will correspond to a new iteration of the Mandelbrot equation. Our input values of c_r and c_i and the values of z_r and z_i are stored as bit size 27, with 4.23 signed fixed point arithmetic, an appropriate range as the Mandelbrot set iterations typically bounded with a range of ± 2 before escape, so the squared magnitude check occurs around 4. With 4.23 fixed-point, we allow for a sufficient integer range to protect against overflows, but additionally, allow for maximum resolution for zooming, accounting the DSP block's ability to do 27x27 bit multiply. Escape detection is implemented through checking whether the square magnitude, which is $|z|^2 = z_r^2 + z_i^2$, is $|z|^2 > 4$, which as mentioned in Section 2, has been mathematically proven as a point that diverges. A small issue regarding overflow errors occurred initially when testing in simulation, which led to an additional check before updating `zr_new` and `zi_new`, to ensure both values of $|z_r|$ and $|z_i|$ are < 2 , which could create overflow if not met, to ensure that proper treatment occurs when escape detection takes place. When $|z|^2 > 4$, our iteration is terminated and iteration count is recorded for the c value alongside a ready flag.

4.1 "Classic" 3 DSP Block Iterator

The baseline Mandelbrot iterator was implemented with a datapath that used 3 DSP blocks per cycle, which each preformed a 27x27 bit multiplication of the 4.23 fixed-point signed values. By evaluating the Mandelbrot set with two separate components, z_r and z_i , separate equation of $z_{r,n+1} = c_r + (z_{r,n}^2 - z_{i,n}^2)$ and $z_{i,n+1} = c_i + 2z_{r,n}z_{i,n}$ were performed in parallel.

These three multiplications were completed using an inferred DSP block using a reusable `signed_mult` module, generating a 54-bit intermediate product then resizing this product through bits [49:23], maintaining the original 4.23 bit signed fixed-point representation.

All values were represented in 4.23 fixed-point representation with 27 bits. This allowed for the use of only 3 27x27 bit multiplies needing to be completed each cycle, fully utilizing DSP blocks to maximize the potential speed and resolution of calculations. This design calculates one Mandelbrot iteration each clock cycle and the three DSP blocks are used to calculate the required products of $z_{r_{new}}^2$, $z_{i_{new}}^2$, and $z_r z_i$. $z_r z_i$ is used in the calculation of $z_{i_{new}}$, which is squared alongside $z_{r_{new}}$.

These squared results are written into registers at the clock edge and; therefore, are available at the next iterative cycle for calculations as z_r^2 and z_i^2 . These values can now be used for calculating new values of z_r and z_i without having to recalculate the squared values which were necessary to compute in order to check if escape conditions have been previously met.

Reset values in this iterator followed that z_r , z_i , z_r^2 and z_i^2 would all be reset to 0 alongside the "counter", "escaped", and "ready" signals. Parameters were set to `MAX_ITERATIONS` = 1000, demonstrating an assumption that if after 1000 iterations has occurred and the escape conditions have not been met, it can be assumed that the Mandelbrot iterator at that coordinate will likely converge. For each iteration that escape conditions or `MAX_ITERATIONS` are not met, values of z_r , z_i , z_r^2 and z_i^2 are assigned, a check for $|z|^2 > 4$ occurs, and values are recalculated.

This 3 DSP block architecture served as the baseline design for this project, as the architecture is simple yet efficient at iterating once per clock cycle and allowing for the possibility of 29 iterators being able to be instantiated and work together in parallel. However, as seen in the following subsections, with clever tactics, less DSP blocks per iterator can be generated at the cost of more clock cycles per iteration.

4.2 Generating Multiple Iterators

Our first attempt at generating the Mandelbrot set using our hardware used only one iterator. However, we were aware that running a single iterator was only a stepping stone, so we were careful to parameterize every aspect relating to the iterators on a local parameter - `NUM_ITERATORS`, which we will call **N**.

We transitioned to two iterators by moving our M10K block instance, iterator instance, and state machine into a generate block with for loop. We also declared arrays of each per-iterator signal so that we would be able index the IO of each iterator using only the corresponding genvar.

4.2.1 M10K Memory Modifications

Further, the M10K module itself was modified so that instances of the module only allocated enough memory to store the number pixels each iterator was responsible for. The equations used to find this number are described below.

$$MEMSIZE = \$ceil(\frac{WIDTH}{N}) * HEIGHT$$

Where `WIDTH` and `HEIGHT` are the screen size in pixels (640x480, respectively). We dispatch pixels to each iterator serially, in step sizes equal to **N** pixels. This means that each iterator handles at most 640 divided by **N** pixels in each of the 480 rows. By allocating this amount of memory for each iterator, we avoided using excess memory and kept the process of muxxing the color input to the VGA relatively simple, which will be discussed later.

4.2.2 Serial Writes and Multiplexed Reads

To keep track of the pixel data computed by each iterator, we developed a system for translating addresses of pixels requested by the VGA to a specific iterator instance i and a local memory address.

Each iterator writes to its unique memory block serially - incrementing the write address by one in most cases. Because some iterators will operate on one less pixel per row than others, we manage the 'x' and 'y' components of the write address independently. The 'x' component refers to the index of the pixel, out of the $\lceil \frac{W}{N} \rceil$ pixels per row that an iterator is responsible for. The 'y' component is similar, but keeps track of the row a pixel belongs to, storing it as a multiple of $\lceil \frac{W}{N} \rceil$. We increment these components every time a new pixel is dispatched to the iterator, keeping them synchronized with the X and Y coordinate of the pixel being computed. Every time this happens, the 'x' component is incremented by 1 (unless moving to the next row), and the 'y' component is incremented by $\lceil \frac{W}{N} \rceil$ (unless wrapping around to the first row). This system allows us to avoid an unnecessary multiply in our datapath (which was part of our critical path at one point) and construct the write address with a single addition.

When the VGA requests a pixel, we now have to decode it's X and Y coordinates to find the memory bank holding its color data and the local address. The memory bank is simple to retrieve, we do it by taking the screen width (640) modulo the current X coordinate. Our dispatch routine was designed specifically to be the reciprocal of the modulo operator, so this just works. The local address is rebuilt in one (VGA_pll) clock cycle using the general formula that we implemented sequentially on the iterator side. We multiply the row (Y coordinate) by the length of a row in each memory bank ($\lceil \frac{W}{N} \rceil$) and add that to the closest multiple of N that is less than the X coordinate ($\frac{X}{N}$). We implemented this equation explicitly, which Quartus was able to synthesize without using any additional DSP blocks. Since this calculation only needs to keep up with the VGA clock (much slower than our M10K PLL), we also were not concerned with the speed. After doing this computation, we're able broadcast the read address to all banks, and multiplex only the result of the desired bank into the VGA pixel data.

See Figures 1 and 2 for a visual representation of the banked memory architecture and dispatch.

		X								
		0	1	2	3	4	5	...	638	639
Y	0	1	2	3	4	1	2	...	3	4
	1	1	2	3	4	1	2	...	3	4
	2	1	2	3	4	1	2	...	3	4
	3	1	2	3	4	1	2	...	3	4
	4	1	2	3	4	1	2	...	3	4
	...	1	2	3	4	1	2	...	3	4
	480	1	2	3	4	1	2	...	3	4

Figure 1: Example diagram of pixel dispatch with 4 iterators on a 640x480 display. Cell contents indicate iterator instance identifier.

(X,Y)	0	1	2	...	159
0	(0,0)	(0,4)	(0,8)	...	(0,636)
1	(1,0)	(1,4)	(1,8)	...	(1,636)
2	(2,0)	(2,4)	(2,8)	...	(2,636)
3	(3,0)	(3,4)	(3,8)	...	(3,636)
...	...	...	...	...	...
480	(480,0)	(480,4)	(480,8)	...	(480,636)

Figure 2: Example of M10K bank for iterator 1 based on example dispatch in Figure 1. Cells represent color data for pixel at (x,y)

4.2.3 Problems

We hit several speed bumps when designing a scalable system for this lab. The first being memory constraints. We discuss our working solution in Section 4.2.1, however our first attempt was more naive. Our first approach was brute force - allocate enough memory for the entire pixel buffer for each iterator, then address pixels directly using their X and Y coordinates. This worked for one iterator, but it does not work for two. There are 307,200 pixels on the VGA screen - each taking up 1 byte in memory. Therefore, one buffer for the entire VGA screen takes up approximately 307KB, or 2,456Kb. Our FPGA only has 397 M10K blocks, making up 3,970 Kb of M10K memory total. If we tried to allocate the entire pixel buffer for each iterator we would quickly exhaust the available resources. As previously discussed, we solved this by allocating only the amount of memory required for each iterator and translating the addresses.

Another obstacle we encountered early on was properly maintaining the Mandelbrot input values (cr and ci) with more than one iterator. Our first attempt involved independently incrementing cr and ci for each iterator without tracking the corresponding X and Y. This meant handling all the incrementing and wrapping independently of the write address. This was a poorly thought out approach, and it presented several interesting bugs. The first visual we managed to display looked similar to TV static (see Figure 3), and it became clear we were losing track of our pixel coordinates somewhere. Our second attempt kept track of the X and Y coordinates plus a starting pixel, but allowed iterators to wrap around to the next row and carry over whatever excess cr that accumulated when incrementing past the end of the range. This generated a, somewhat interesting, skewed Mandelbrot set (see Figure 4). After this, we came up with the refined dispatch method discussed in Section 4.2.2 and generated a properly oriented Mandelbrot.

One other bug we encountered during this phase was several columns of pixels appearing to wrap from the right side of the VGA display to the left. The problem turned out to be a one-off issue with our write address translation, caused by incrementing the write address one cycle too early. We solved this by registering the correct write address on the cycle that the



Figure 5: Mandelbrot graphical bug with 87 iterators at 150MHz.

pixel is computed. Although this problem stuck around for several days, we unfortunately did not get a photo of it.



Figure 3: TV static created by first attempt at pixel dispatch.

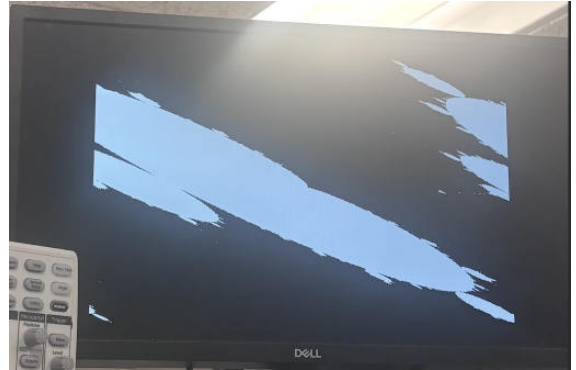


Figure 4: Skewed Mandelbrot set generated from a second, still undercooked, dispatch routine.

Graphical labs are very interesting because there is a high chance that any bugs in the software/hardware result in something that is visually interesting. Another bug that produced an intriguing visual result was excessive overclocking. After implementing the 3-cycle design discussed in Section 4.3.2, we were ambitious about the frequency capabilities of our new design. Naturally, we cranked the iterator/M10K clock up to 150 MHz and observed what you see in Figure 5

4.3 Attempted Reduced-DSP Iterator Optimization

We attempted to speed up generation of the Mandelbrot set by increasing parallelism. We designed two variations on the standard 3-DSP iterations, taking inspiration from multi-cycle FSM designs.

4.3.1 Two DSP - Two Cycle Iterator

Our first attempt at increasing parallelism uses only two DSP blocks, utilizing a hybrid-FSM design. One DSP handles calculation of $Z_R^{N+1^2}$ while the second computes $Z_R^N * Z_i^N$ in the first cycle and $Z_i^{N+1^2}$ in the second cycle. The motivation behind this design was to eliminate the critical path that passed through two DSP blocks. The goal was to increase the clock frequency by only doing one multiply per cycle and increase the total number of iterators running in parallel.

We achieved a modest increase in clock frequency as a result (10 MHz) but a net slow-down in Mandelbrot generation. With 43 iterators (using 86/87 DSP blocks on the DE1-SOC) we generated the full Mandelbrot set in 20 ms, approximately 6 ms slower than our original 3 DSP design.

This design has several obvious flaws in retrospect. Firstly, we were only able to achieve a $\sim 10\%$ increase in clock frequency and roughly a 50% increase in the number of iterators running. The tradeoff for these improvements was a 100% increase in the number of cycles required per iteration (from 1 to 2). Had we been able to achieve closer to 100% increase in clock frequency, this design may have had a chance at providing real improvement - however, the critical path in the modified design laid in a long chain of combinational logic spanning from *cr_in* on the Avalon Bus to the output register of the multiplexed multiplier. This path bottle-necked our frequency gains and indicated that a pipelined design may have been the better route for optimization. Secondly, because two is not a factor of 87, we were not able to utilize all 87 DSP blocks on the FPGA (with 43 iterators we used only 86). This obviously is sub-optimal as it means we weren't able to utilize a portion of the FPGAs compute.

4.3.2 One DSP - Three Cycle Iterator

Our second attempt at increasing parallelism stayed with the multi-cycle philosophy, but attempted to fix the flaws in the two-cycle design by fully committing to the FSM design rather than a hybrid design. Our main goal was to 3X the number of iterators we could run relative to the base design and keep the moderate increase in clock frequency. We wanted to maintain the same ratio of iterators to cycles-per-iteration as the baseline design, something that we failed to do in the two-cycle design. In theory, a design with 87 3-cycle iterators will compute the Mandelbrot set at the same speed as a design with 29 1-cycle iterators - all other factors held constant. Therefore, if we could sustain an increased clock frequency, we would see some gains.

While this design did yield some improvement over the two-cycle design, it still failed to surpass the base design. With 87 iterators, we could generate the whole Mandelbrot set in ~ 18 ms. A postmortem on this design also reveals several flaws in the multi-cycle principle.

Firstly, instantiating 87 iterators ballooned the area and routing requirement of the total design. On the first compile of this design, the Quartus fitter failed routing after roughly an hour of repeated attempts with "increased optimization." To get around this, we had to slightly tweak the Quartus compiler - telling it to prioritize constraining area rather than performance (speed of compilation). This tweak allowed the Fitter to succeed with 87 iterators and 81% logic utilization on the FPGA. While this is not inherently a flaw in this

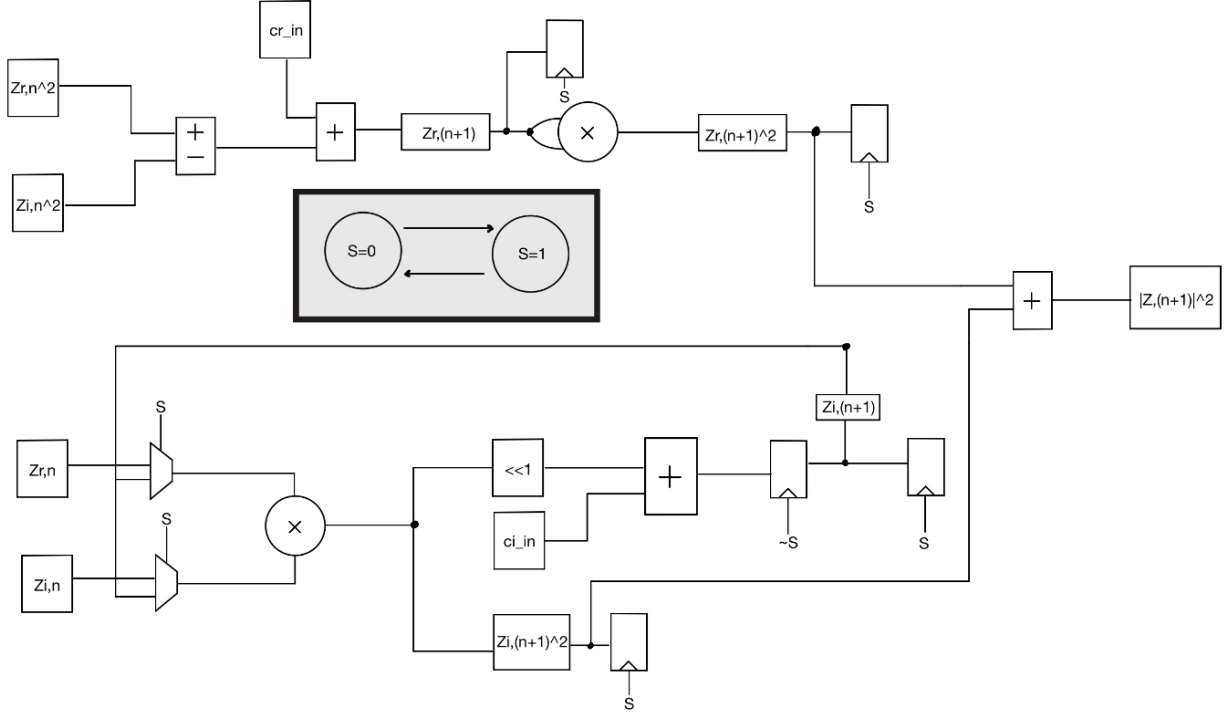


Figure 6: Block Diagram of two-cycle hybrid FSM Design. Including State Diagram and control signals.

design, it makes it clear that this design likely has much longer critical paths than previous designs - largely due to congested routing on the FPGA. Unfortunately, this turned out to be true and we were unable to meet timing with the 130 MHz clock that worked on our two cycle design. We reduced the clock to 120 MHz (the frequency of the base design) and achieved the 18 ms figure discussed previously.

This still leaves approximately 4 ms of time loss unaccounted for - if our working theory was correct. We find it likely that tweaks to the Quartus compiler and/or flaws in the load balacing of our design account for this discrepancy. While still unclear exactly what caused this time loss, this experiment combined with our discussions with other groups reinforces our conclusion that pipelining would have been the more optimal design philosophy for this problem.

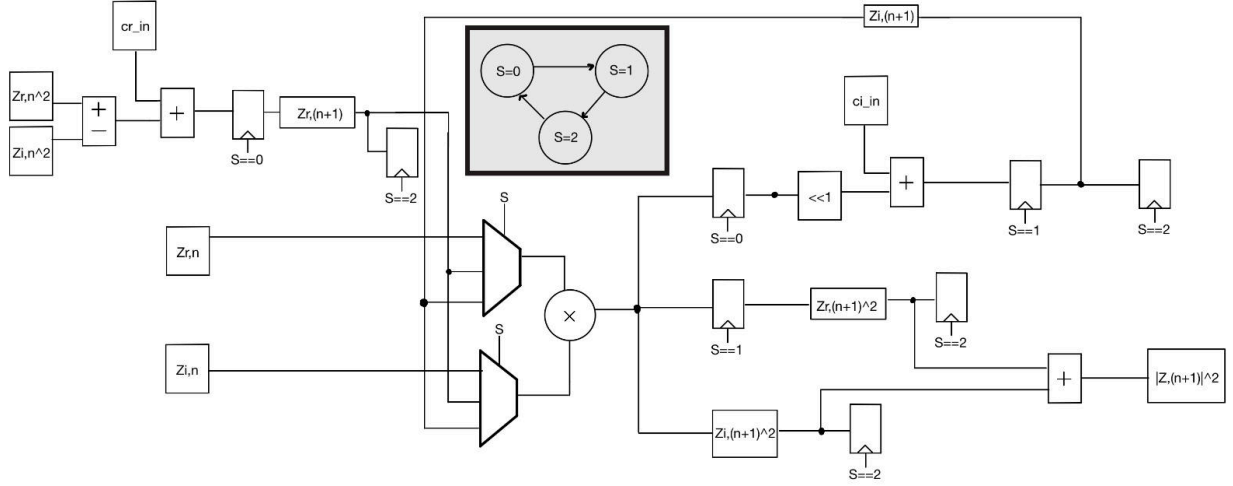


Figure 7: Block Diagram of three-cycle, single DSP iterator design. Including State Diagram and control signals.

5 Design and Testing Methods

5.1 Hardware Testing

The primary form of hardware testing used to test the Mandelbrot Verilog design was through comparison with a Python script which calculated the number of iterations before divergence of the Mandelbrot set iterator. Both the hardware and software implementations, the system iteratively calculates $z_{n+1} = z_n^2 + c$, with constant initial condition $z_0 = 0$. While the Python script used floating point arithmetic compared to the 4.23 fixed-point arithmetic in the hardware implementation, given matching values for c_r and c_i , the number of iterations before divergence remained equal between the Python implementation and hardware testing. Additionally, to check the values of the intermediate calculations in Python vs Verilog, the floating point values in python were scaled by 2^{23} so that the floating point values matched the fixed point values and comparison of intermediate values of z_{n+1} can be compared.

To test the system, a non-synthesizable Verilog testbench was created, defining as our device under test (DUT) as our Mandelbrot iterator, and initializing a clock and a reset signal before inputting scaled values of c_r and c_i to match 4.23 fixed-point format. Once reset was enabled and input values were defined, the system began iteratively computing new values of z_r and z_i each clock cycle, while determining if the escape condition has been met, so that the iteration counter can be set as ready.

An example waveform sourced from additional testing on Icarus Verilog waveform viewer is shown below, where these input values of c_r and c_i are shown alongside the counter value incrementing until ready is asserted and intermediate calculations occurring. Through comparison of both the counter values and the intermediate values with the Python scripts detailed below allowed for confident verification of our Mandelbrot iterators.

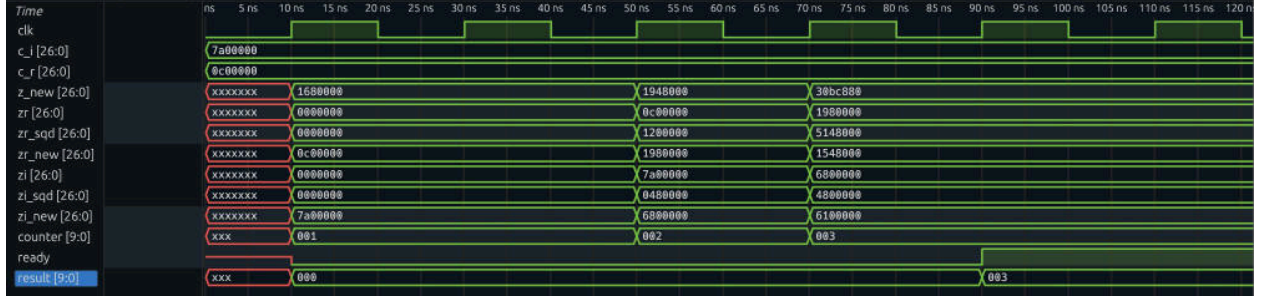


Figure 8: Waveform of Single Iterator

Additionally, to ensure consistency between the software implementation of the Mandelbrot solver and the dedicated hardware module, a variety of coordinates were tested including those that were determined to diverge rapidly, converge, and gray areas that may or may not converge, ensuring that edge cases of c_r and c_i values are tested properly. An example of such case could be when $c_r = -1.99$, and $c_i = -1.49$, the bottom left corner of the Mandelbrot set, which is expected to diverge incredibly rapidly.

Another example of a convergent case occurs when $c_r = -0.65$ and $c_i = 0.88$. Such a case's iteration projection in the complex plane is shown in the below figure, of which's code was sourced from [1]. In addition to the provided viewer, which allowed us to visually spot interesting edge cases and cases of extreme convergence and divergence, a separate Python script was used, which provided the values for $|z| * 2^{23}$, allowing for cross referencing with ModelSim.

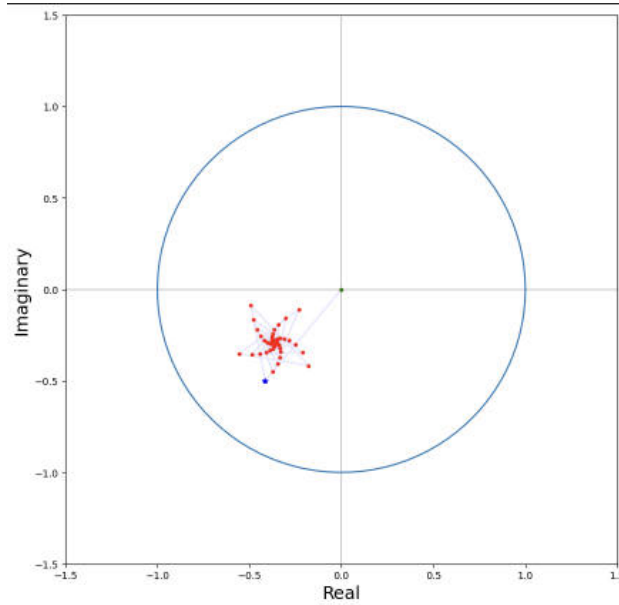


Figure 9: An Interesting Case of Convergence for the Mandelbrot Iterator

Finally, to confirm the correctness of the number of iterations before convergence, a third Python script was run for each selected value, providing the maximum number of iterations

before divergence of the Mandelbrot set, for true cross referencing of the iteration counter value of the Verilog module when the ready signal was enabled.

Outside of the combined system, small modules such as our signed multipliers were tested prior to ensure proper concatenation occurred of the 54 bit long multiply intermediate. Additionally, the waveforms of each signal were verified by comparing the output values with the expected values for each value tested.

Overall, the Mandelbrot hardware design was validated through comparison with a Python reference model, waveform inspection in ModelSim, and visual verification of correct behavior for signals such as ready and reset. A variety of input conditions were tested and both intermediate values and expected return values were both verified, allowing for confidence in the system generation at scale.

5.2 Software Testing

The software testing for this lab involved verifying that our mouse control was functioning properly, the HPS was writing to the correct memory addresses on the Avalon bus, and writing the correct values in fixed point form. We our mouse control using print statements which we could view over SSH. We verified that when the mouse was moved we received the expected feedback from our C program. In doing this, it became clear that we were properly interfacing with the mouse.

Verifying that we were writing the proper values to the proper memory addresses involved mostly visual inspection, and some print statements to reveal the internals. Every time our C loop runs, it reads back the values written to the FPGA - `cr_start`, `ci_start`, `cr_stop`, and `ci_stop` - and prints them to the console. We convert these values from fixed point to floating point and format them as ordered pairs representing the four corners of our viewport. This allowed us to verify that when our system resets, it does indeed show the Mandelbrot set in the range (-2.0, -1.0) to (1.0, 1.0). We also verified that when we zoomed and panned, our ranges changed as expected. Additionally, we had to verify that when we zoomed or panned from the HPS, we were telling our hardware to restart the render timer. We also read the timer output from our hardware and printed it to the console. Every time we clicked the mouse to zoom/pan we observed the timer change to represent how quickly the solver was able to render that portion of the set.

To verify that the signal being sent from the FPGA to the HPS was valid (the HPS reset signal) we verified that when KEY0 was pressed, our C program reacted accordingly and reset the viewport to the default range.

6 Results

In this lab, we achieved a best time of 14 ms to compute the Mandelbrot set in the default range. We did this with 29 iterator instances, utilizing all 87 DSP blocks on the DE1 FPGA.

Additionally, we designed a system that is scalable up to a theoretical 639 iterators (after which some minor tweaks would be required) given an FPGA with sufficient hardware resources. We managed to compile and run a design with 87 3-cycle iterators working

concurrently to generate the Mandelbrot set - exhausting the routing resources of the DE1 FPGA.

During this lab we learned about hardware limitations and the trade offs of multi-cycle architectures. The results of this lab are further discussed in the following sections, notably sections 7 and 8.

7 Chip Planner Layout

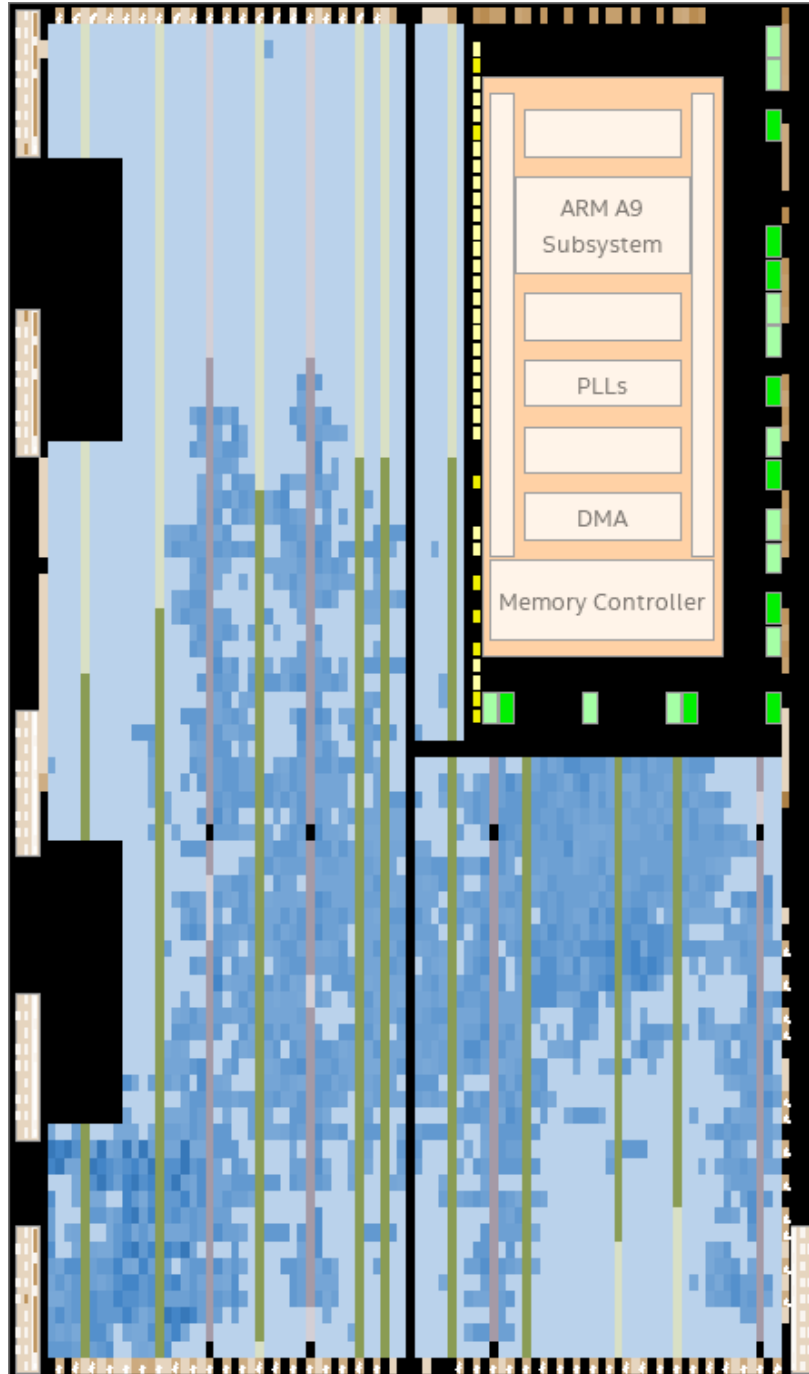


Figure 10: FPGA Chip Planner showing resource utilization of the Mandelbrot accelerator.

Figure 8 shows the Chip Planner view generated by Quartus, which provides a physical representation of how the Mandelbrot accelerator is mapped onto the FPGA fabric of the DE1-SoC. Unlike the RTL schematic, which primarily demonstrates the logical connectivity of the system, the Chip Planner shows the physical placement of logic elements, embedded

memory blocks, and routing resources after synthesis, placement, and routing have been completed in Quartus.

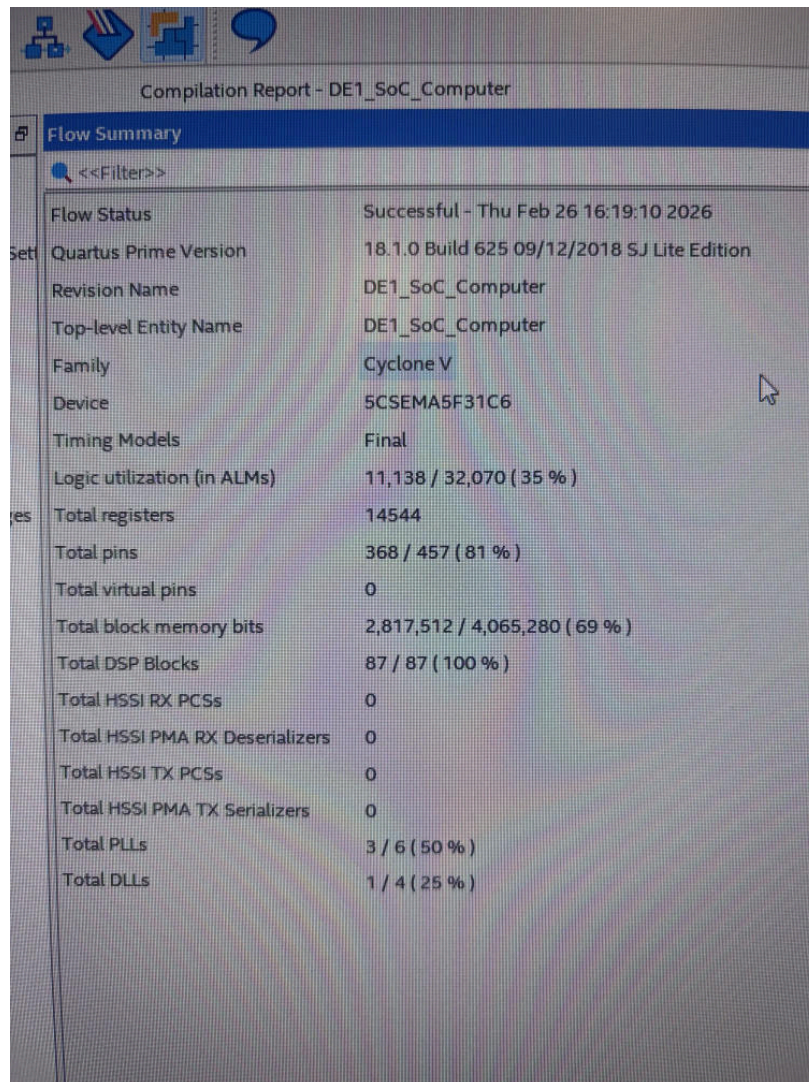
The large rectangular region on the right side of the planner corresponds to the Hard Processor System (HPS), which contains the ARM A9 subsystem, PLLs, DMA controller, and memory controller. This region is fixed silicon on the SoC, meaning it is not part of the programmable FPGA fabric. The remaining portions of the figure represent the programmable logic area where our Mandelbrot accelerator hardware was implemented.

The blue regions across the FPGA indicate areas where logic resources such as adaptive logic modules (ALMs) and registers are utilized by the design. In our Mandelbrot system, these resources correspond to the iterator pipelines, pixel dispatch logic, arithmetic datapaths, and VGA interface modules. The green vertical columns represent embedded memory resources such as M10K blocks, which are used in our design to store the pixel color values computed by the iterator pipelines before they are read by the VGA controller for display.

From the layout, we can observe that a large portion of the FPGA fabric is utilized, and the synthesis and placement tools distributed the logic resources across a wide region of the chip rather than concentrating them in a single area. This distribution helps reduce routing congestion and improves timing performance by keeping related logic and memory blocks relatively close together. The routing paths visible throughout the device illustrate how signals move between the iterator pipelines, memory blocks, control logic, and the VGA subsystem.

Poor placement could lead to long routing paths, increased propagation delays, and potential timing violations. However, as shown in the Chip Planner in Figure 8, the Mandelbrot accelerator successfully maps to the FPGA hardware, and the logic and memory resources are efficiently distributed across the fabric. This layout supports the highly parallel computations required for the fast fractal rendering achieved in our design.

8 Hardware Resource Utilization



Flow Summary	
Flow Status	Successful - Thu Feb 26 16:19:10 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	DE1_SoC_Computer
Top-level Entity Name	DE1_SoC_Computer
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	11,138 / 32,070 (35 %)
Total registers	14544
Total pins	368 / 457 (81 %)
Total virtual pins	0
Total block memory bits	2,817,512 / 4,065,280 (69 %)
Total DSP Blocks	87 / 87 (100 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	3 / 6 (50 %)
Total DLLs	1 / 4 (25 %)

Figure 11: Hardware Utilization

Quartus Prime reports that our synthesized Mandelbrot accelerator uses 11,138 adaptive logic modules (ALMs) out of 32,070 available (35%), along with 14,544 registers. The design also uses 2,817,512 block memory bits out of 4,065,280 (69%), and 87 DSP blocks out of 87 available (100%). In addition, the system utilizes 368 I/O pins out of 457 (81%), 3 PLLs out of 6 available (50%), and 1 DLL out of 4 available (25%).

These hardware utilization statistics reflect the architecture implemented for our Mandelbrot visualization system, which relies on a large number of parallel iterator pipelines implemented in FPGA hardware. Each iterator continuously evaluates the Mandelbrot iteration equation using fixed-point arithmetic and performs several multiplications and additions every clock cycle. We intentionally structured the design to maximize parallelism, which required many hardware arithmetic units to be instantiated simultaneously.

The ALM usage primarily comes from the control logic, iterator pipeline logic, pixel dispatch circuitry, and VGA interface modules. Each iterator pipeline contains arithmetic datapaths, iteration counters, comparison logic for divergence testing, and control state machines that determine when a pixel has finished computing. In addition, the VGA subsystem requires logic resources to generate display timing signals, manage pixel coordinates, and multiplex the outputs from multiple iterator pipelines to the display memory.

The large number of registers used in this design is expected because the Mandelbrot computation and VGA display logic are fully synchronous. Registers are required to store intermediate arithmetic values within the iterator pipelines, maintain iteration counters, track pixel coordinates, and hold pipeline stage values used in the arithmetic datapath.

The DSP block usage is especially significant in this design, as all 87 available DSP blocks are utilized. These DSP blocks implement the fixed-point multiplications required for the Mandelbrot iteration, such as computing z_r^2 , z_i^2 , and $2z_r z_i$. Quartus maps these operations to dedicated hardware multipliers because they are faster and more efficient for multiplication than implementing these operations using general-purpose logic elements. The full utilization of DSP resources reflects the highly parallel nature of the design and the large number of iterator pipelines instantiated to maximize computational throughput.

The block memory utilization of 69% is primarily associated with storing the pixel color values produced by the iterator pipelines. Each iterator writes the computed color value of the pixels it processes into on-chip M10K memory blocks. These memory buffers allow the system to store intermediate pixel results while the VGA controller reads the data to display the Mandelbrot image on the screen.

Finally, the I/O usage is relatively high because the system interfaces with multiple external components on the DE1-SoC board, including VGA output, control inputs, and communication signals between the FPGA and the HPS. The PLLs are used to generate the required clock domains for the system logic and the VGA display timing used during rendering.

8.1 Critical Path

For our Mandelbrot accelerator system, we expect the longest combinational logic chain to occur within the arithmetic datapath of the iterator pipelines.

Each iterator repeatedly evaluates the Mandelbrot iteration equation

$$z_{n+1} = z_n^2 + c$$

using fixed-point arithmetic. During each iteration, the system computes the real and imaginary components of the next complex value using the expressions

$$z_{r,\text{new}} = z_r^2 - z_i^2 + c_r$$

$$z_{i,\text{new}} = 2z_r z_i + c_i$$

To perform these computations, the system requires several arithmetic operations within a single clock cycle, including multiple multiplications, additions, and subtractions. In particular, computing z_r^2 , z_i^2 , and $2z_r z_i$ requires fixed-point multiplications that are mapped by

Quartus onto dedicated DSP blocks. Although these multipliers are implemented in specialized hardware, the surrounding logic requires additional operations such as sign extension, truncation, shifting, and addition, which all contribute to the overall combinational delay.

Therefore, the critical path most likely occurs along the register-to-register path that begins at the current complex state registers storing z_r and z_i , passes through the arithmetic units responsible for squaring and multiplying these values, and ends at the registers that store the updated values for the next iteration.

A representative path in the datapath can be summarized as:

state register $\rightarrow$ multiplication $\rightarrow$ subtraction/addition $\rightarrow$ truncation/shift $\rightarrow$ next state register

Because multiple arithmetic operations may be chained together within the datapath, this path contributes to the longest propagation delay in the design. Therefore, the efficiency of the arithmetic pipeline plays a crucial role in determining the timing performance of the system. Understanding the critical path allows us to ensure that the design operates within the desired clock frequency range. If the chain of arithmetic operations becomes too long, the maximum achievable clock speed would decrease, which would limit the overall performance of the Mandelbrot accelerator.

By utilizing dedicated DSP multipliers and structuring the arithmetic datapath efficiently, our design is able to maintain a clock frequency that supports fast parallel computation across all iterator pipelines.

8.2 Program Listings

For this subsection of Verilog design, the 3 DSP block iterator will be primarily considered as a result of the 2 and 1 DSP block iterators failing to provide significant improvements in Mandelbrot generation times. The following table describes each of the primary modules used to generate the Mandelbrot Set Visualizer.

Table 1: Program Listing Table: Verilog Mandelbrot System Design

Module	Type	Description
<code>testbench_mandelbrot</code>	Testbench	Testbench module for simulation. This testbench generates clocks, applies reset sequencing, and initializes inputs the Mandelbrot iterator for Testing. This test bench instantiates <code>mandlebrot_system</code> as the DUT and allows for testing of the synthesizable <code>mandlebrot_system</code> module.
<code>signed_mult</code>	RTL (Combinational)	Signed fixed-point multiplier for the 4.23 format used for Mandelbrot iterations. Computes the full, 54 bit product, <code>mult_out = a*b</code> and truncates bits [49:23] to return the correctly scaled 27-bit fixed-point result.
<code>mandlebrot_system</code>	RTL (Datapath + State)	Implements a Mandelbrot iterator using the iteration $z_{n+1} = z_n^2 + c$. Computes the real and imaginary updates and the square of those updates, tracks the number of iterations, performs escape detection for the condition $ z ^2 > 4$, and outputs escape status and iteration result.
<code>M10K_1000_8</code>	Memory (On-chip RAM)	Per-iterator memory block used to store pixel information such as color data. Each iterator writes computed pixel values to its own M10K piece and this is later fed to VGA display.
<code>vga_driver</code>	RTL (Display Interface)	Generates VGA timing and synchronization signals to send to VGA monitor, reads pixel color data from M10K memory, and outputs RGB signals for VGA display.

The system generates a visualization of the Mandelbrot set by evaluating the escape time as described by number of iterations until the escape condition is met. For each coordinate on the screen, the `mandlebrot_system` iterates until either $|z|$ exceeds the escape threshold or `MAX_ITERATIONS` are reached. The resulting iteration count is then mapped to an 8 bit color value, stored in on-chip M10k memory and fed through VGA display driver to read these pixel values and display them on the VGA display. Points that do not diverge within

1000 iterations are colored black and as points escape, a gradient of colors are assigned, corresponding to the escape time, which produces the Mandelbrot's characteristic gradient patterns.

8.3 User Interface

The Hard Processor System (HPS) managed the user interface for the Mandelbrot visualizer using a C program that communicates with the FPGA through memory-mapped registers. The HPS software allows the user to interactively explore different regions of the Mandelbrot set by modifying the coordinates of the complex plane being rendered. These parameters are continuously written to control registers, which the FPGA Mandelbrot accelerator reads in order to recompute the fractal.

The C program accesses FPGA registers using Linux memory mapping through `/dev/mem`. By mapping the lightweight HPS-to-FPGA bridge into user space, the program obtains pointers to hardware registers that control the Mandelbrot rendering parameters. These registers contain the starting and ending coordinates of the real and imaginary components of the complex plane, as well as the step sizes used to move between adjacent pixels. During execution, the program continuously updates these values and writes them to the FPGA so that the hardware iterators can compute the Mandelbrot set for the selected region.

To allow interactive control, the program creates a separate thread that monitors keyboard input using the `ncurses` library. This thread allows the user to dynamically modify the viewing region of the Mandelbrot set. For example, when the user presses the `i` key, the current range of the complex plane is reduced to perform a zoom-in operation, while pressing the `o` key increases the current range to zoom out. Additional key commands allow the user to pan across the fractal: `w` and `s` shift the view vertically, while `a` and `d` move the view horizontally across the complex plane.

When the user adjusts these parameters, the program recalculates the corresponding step sizes for the real and imaginary axes and writes them to the FPGA control registers. The FPGA Mandelbrot accelerator then recomputes the fractal image using the updated coordinate range. Because the hardware iterators operate in parallel, the system can render the updated fractal view very quickly after each interaction.

The interface also provides performance feedback to the user. The FPGA records the number of clock cycles required to render the Mandelbrot set and stores this value in a hardware register. The HPS reads this value and converts it into milliseconds using the known clock frequency of the FPGA. This rendering time is displayed both in the terminal and on the VGA display, allowing users to observe the performance of the hardware accelerator while exploring the fractal.

Table 2: Program Listing Table: Mandelbrot Software Control Program

Module	Type		Description
<code>control_thread</code>	Software	Thread	This is the keyboard control thread that we used for interactively navigating the Mandelbrot set. This thread uses the <code>ncurses</code> library to allow reading user key presses and adjusting the viewing window of the fractal by altering <code>cr_range</code> , <code>ci_range</code> , <code>cr_start</code> , and <code>ci_start</code> . This also supports zooming and panning across the complex plane while the FPGA continues to render.
<code>main</code>	Software Control / Interface	Con- Host	Top-level host program responsible for communication with the FPGA Mandelbrot solver. Opens <code>/dev/mem</code> , maps the lightweight HPS-to-FPGA bridge into the virtual memory, initializes pointers to the memory-mapped control registers, launches the thread for control, and writes updated Mandelbrot viewing coordinates to the memory so the FPGA can iterate. Additionally, it reads the FPGA timing counter and returns the rendering time for display on the VGA display.
Fixed-point conversion macros	Software	Utility	Set of macros including <code>float2fix</code> , <code>fix2float</code> , <code>int2fix</code> , <code>fix2int</code> , <code>multfix</code> , and <code>divfix</code> . These provide conversion support for the 4.23 fixed-point representation used on the FPGA for Mandelbrot iterations. This allows for the software to correctly generate the scaled parameters necessary for proper interfacing with the FPGA through memory-mapped registers.

8.4 Documentation and Answers to Lab Questions

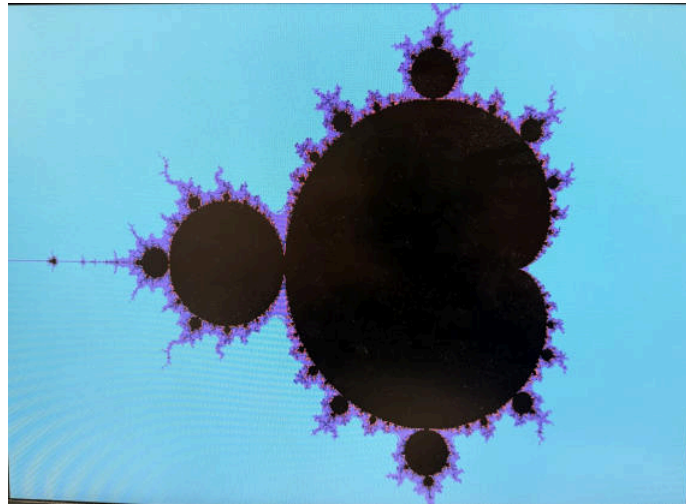


Figure 12: Rendered Mandelbrot set produced by our FPGA hardware accelerator.

8.5 A collection of photographs

For us, two areas on the Mandelbrot set stand out. These are the entrance to seahorse valley (Figure 13) and the mini Mandelbrot (Figure 14)

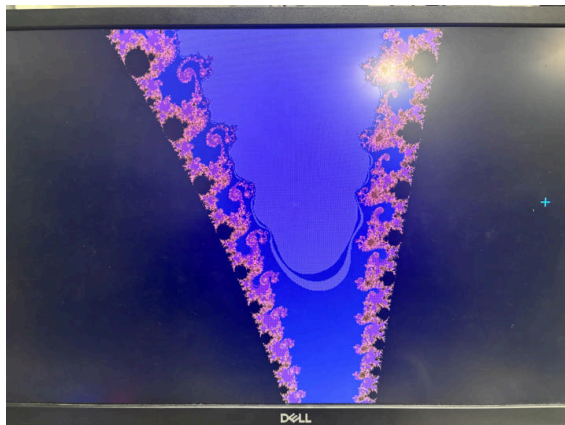


Figure 13: Entrance to "seahorse valley." A popular part of the Mandelbrot set. Range: $x=[-0.87, -0.68]$ $y=[-0.26, -0.13]$



Figure 14: Mini version of the Mandelbrot set. Lives to the right of the main set, and much smaller! Range: $x=[-1.48, -1.46]$ $y=[-0.009, 0.006]$

8.6 Timings for Various Regions

Table 3: Computation Times for Mandelbrot Set Regions

Requested Region	Actual x Range	Actual y Range	Time (ms)
$x \in [-2, 1], y \in [-1, 1]$	$[-2, 1]$	$[-1, 1]$	16.3
$x \in [-0.758, -0.75], y \in [0.05, 0.06]$	$[-0.759, -0.747]$	$[-0.047, -0.4]$	64
$x \in [-1.45, -1.3], y \in [-0.07, 0.07]$	$[-1.49, -1.3]$	$[-0.06, 0.06]$	25

8.6.1 Platform Designer

For this lab we configured several PIO ports on the AXI bus for communication between the HPS and the FPGA.

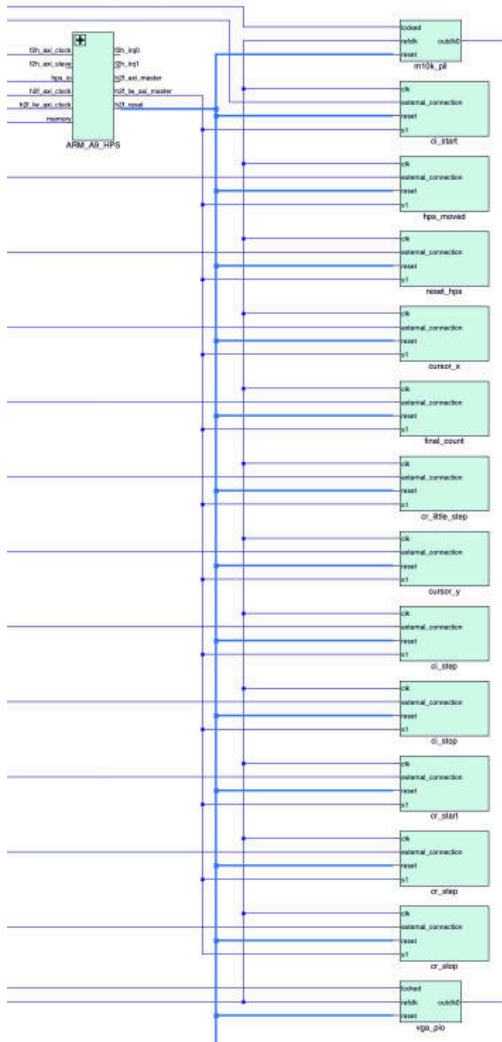


Figure 15: Quartus generated block diagram of the modules configured on the AXI Bus via Platform Designer.

- **cr_start:** Output, real component of the initial value for c .
- **ci_start:** Output, imaginary component of the initial value for c .
- **cr_stop:** Output, real component of the end value for c .
- **ci_stop:** Output, imaginary component of the end value for c .
- **cr_step:** Output, real component of step size between each pixel computed by an iterator.
- **ci_step:** Output, imaginary component of step size between each pixel computed by an iterator.
- **cr_little_step:** Output, real component of step size between each adjacent pixel in a row.
- **cursor_x,y:** Output, X and Y components of the mouse cursor on the VGA screen.
- **hps_moved:** Output signals FPGA to restart computation and timing when the viewport is moved.
- **hps_reset:** Input, signals HPS to reset all outputs to default states when FPGA is reset (KEY0).

Our QSYS design primarily facilitates communication from the HPS to the FPGA. Since all of the computation and VGA interaction was done from the hardware side, our C program was only responsible for sending over the initial conditions to generate and other UI necessities like the mouse cursor location. When the HPS detects a mouse click, it computes the new region for the viewport (a start point and a range), calculates an end point, and sends the viewport size to the FPGA. Then, the HPS tells the FPGA that it has requested a view change and tells it to reset using the `hps_moved` signal. As seen above, most of the PIO on the bus reflect this. The one exception is `hps_reset`, which tells the HPS when the FPGA has been reset (KEY0) so that it can reset to the default range. To ensure that this value is read properly from the C program, its declared as a volatile 8 bit integer to prevent the compiler from optimizing operations on this variable with caching.

9 Comments

This lab helped us understand the challenges involved in designing a hardware accelerator on an FPGA and how hardware parallelism can significantly improve performance for computationally intensive problems such as rendering the Mandelbrot set. Even though the mathematical formula for the Mandelbrot iteration may not seem difficult to implement, implementing it efficiently in hardware required careful consideration of the datapath design, memory organization, and parallel iterator pipelines. A large portion of our time was spent debugging, refining, and optimizing the system rather than simply implementing the basic iteration equation.

One of the most difficult challenges we encountered involved fixed-point arithmetic. While the Mandelbrot algorithm is straightforward when implemented in floating-point software, hardware implementations require fixed-point representations that must be carefully scaled. Small mistakes in scaling, bit shifting, or truncation often caused incorrect fractal patterns or unexpected divergence behavior. Debugging these issues required examining intermediate signals and verifying that the arithmetic operations were producing the expected results.

Another challenge was designing a parallel architecture that maximized the FPGA resources available while maintaining correct memory access patterns. Because multiple iterator pipelines compute pixels simultaneously, we had to carefully manage how pixel coordinates were distributed across the iterators and how the results were stored in memory. Ensuring that each iterator wrote to the correct memory location without conflicts required careful address calculations and memory organization. This process highlighted how important data flow and memory management are when designing high-performance hardware systems.

We believed that this lab was an amazing way to really get us thinking on how to optimize our designs where we reduced the number of logic parts needed in our design but to also exhaust the FPGA resources and take advantage of its capabilities.

10 Additional Questions

10.1 What determines how deep you can zoom?

The determining factor for the maximum depth primarily comes from the precision of our fixed-point numeric representation, but additionally, resolution is limited by our maximum amount of iterations and the VGA display resolution. Our design uses 4.23 fixed-point representation, where 23 bits are used for fractional precision. This means that the smallest possible step that can be represented is $\Delta \approx 2^{-23}$. Due to this, as we zoom in deeper than this precision allows, the nearby coordinates are numerically identical and the fractal can no longer be detailed. Additionally, as a fault of our fixed-point representation, at very high zoom levels, rounding errors from the fixed-point arithmetic accumulate leading to the possibility of divergent trajectories at very high levels of resolution. Additionally, our iteration limit of MAX_ITERATIONS, which in our iterator was set to 1000 could lead to resolution issues in terms of color, where we only had a few colors in a basic gradient that with more iterations and additionally more colors, could lead to higher resolution of our pixels in terms of color and better resolution in areas of interest where there is a region that has adjacent points with some diverging and some not-diverging in our interval. This will reveal more detailed boundaries. Lastly, our VGA display had resolution of 640x480, which allows for more visible detail while zooming but does not allow for additional numeric precision.

The depth to which the Mandelbrot set can be explored is primarily limited by the precision of the fixed-point representation, which in this design was 4.23 fixed-point which provides 2^{-23} resolution in the complex plane. To improve the possible depth that can be explored, the fixed-point representation can consist of more fractional bits; however, this would lead to a slower computation due to more computational resources being required.

11 AI Usage

We used artificial intelligence very limitedly, as we only used it to clarify our understanding, polish our work, and debug any mistakes or errors we were facing. For example, through the development process, AI was occasionally used to help interpret compiler and synthesis error messages and to assist in diagnosing issues that came up during debugging. Specifically, it was used to help identify syntax mistakes, resolve signal width mismatches and understand the potential causes for any incorrect logic in our hardware compilation and testing.

AI was also utilized as an aid when reviewing hardware and software implementation details. In particular, it helped clear up aspects of the Verilog design process, memory-mapped communication between the HPS and FPGA, and other system-level concepts when some unexpected behavior had occurred during development. When teaching assistants or professors were not present when we were working outside designated lab hours, AI served as a teaching assistant to clarify any information from the checkoff requirements that seemed confusing or required more context to understand. Moreover, we used AI to describe the functionality of several sample code that was provided to us such as the code for utilizing the mouse to zoom in and out of our design.

Furthermore, AI's role in writing this report was to polish our language by fixing any grammatical and spelling mistakes and modifying the flow of our sentences in our initial drafts written by us. All the information describing our implementation of this lab written in this report was developed with our own ideas and thoughts but was strengthened through the use of AI to allow clarity for the reader.

Prompt Interaction Log

Session Date: February 27, 2026

Topic: Understanding Linux mouse input code for HPS-FPGA design integration

Prompt #	Date	Prompt Summary	Edits	Insertions	Deletions	Mods Accepted
1	Feb 27, 2026	User requested an explanation of a C program that reads raw mouse input from /dev/input/mice.	0	0	0	0
2	Feb 27, 2026	User asked which sections of the code should be modified for their FPGA/HPS system design without providing code changes.	0	0	0	0
3	Feb 27, 2026	User requested generation of a prompt log including timestamps and counts of edits and code changes.	0	0	0	0

Table 4: Prompt interaction log for discussion regarding Linux mouse input and HPS–FPGA integration.

Session Summary

- Total Prompts: 3
- Total Edits Suggested: 0
- Total Code Insertions: 0
- Total Code Deletions: 0
- Total Code Modifications Accepted: 0

During this session, the interaction focused on understanding low-level Linux device input via `/dev/input/mice`, identifying which sections of the program would be modified for integration with an HPS-FPGA co-design, and generating a documentation-style prompt log. No direct code edits or modifications were requested or applied.

References

- [1] Van Hunter Adams, *ECE 5760: Mandelbrot Set Visualizer Lab*, Cornell University. Available: <https://vanhunteradams.com/DE1/Mandelbrot/Mandelbrot.html> Accessed: March 2026.

11.1 Program Listings

```
1
2 #include <pthread.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <fcntl.h>
7 #include <sys/types.h>
8 #include <sys/ipc.h>
9 #include <sys/shm.h>
10 #include <sys/mman.h>
11 #include <sys/time.h>
12 #include <math.h>
13 #include <stdint.h>
14 // #include <curses.h>
15 // this includes new zooming technique
16 // gcc mandelbrot.c -o mb -O2 -lm -pthread -lncurses
17
18 #define SCALE 8388608 // 2^23
19 #define CLK_SPEED 120000000.0
20
21 /* Cyclone V FPGA devices */
22 #define HW_REGS_BASE          0xff200000
23 // #define HW_REGS_SPAN       0x00200000
24 #define HW_REGS_SPAN          0x00005000
25
26 #define CR_STEP_OFFSET 0x10
27 #define FINAL_COUNT_OFFSET 0x20
28 #define CR_ITER_STEP_OFFSET 0x30
29 #define CI_ITER_STEP_OFFSET 0x40
30 #define CR_START_OFFSET 0x50
31 #define CI_START_OFFSET 0x60
32 #define CR_END_OFFSET 0x70
33 #define CI_END_OFFSET 0x80
34 #define CURSOR_POSX_OFFSET 0x90
35 #define CURSOR_POSY_OFFSET 0x100
36 #define HPS_RESET_OFFSET 0x110
37
38
39 typedef int32_t fix23;
40 #define multfix(a,b) (((fix23))(((( signed long long a))*((signed
    long long b)) >> 23))
41 #define divfix(a,b) (fix23)((( signed long long)(a) << 23 / (b)))
42 #define int2fix(a) ((fix23))((a) << 23))
43 #define fix2int(a) ((int))((a >> 23))
44 #define float2fix(a) (fix23)(a * SCALE)
45 #define fix2float(a) (float)(a) / SCALE
```

```

46
47 #define NUM_ITERATORS 87
48
49
50 // these are global var from lorenz.c file
51 #define FPGA_CHAR_BASE          0xC9000000
52 #define FPGA_CHAR_SPAN          0x00002000
53
54 // lorenz.c
55 void *vga_char_virtual_base;
56 volatile unsigned int *vga_char_ptr = NULL;
57
58 void *h2p_lw_virtual_base;
59
60 // Outputs
61 volatile fix23 *cr_step_ptr;
62 volatile fix23 *cr_iter_step_ptr;
63 volatile fix23 *ci_iter_step_ptr;
64 volatile fix23 *ci_start_ptr; // Value at which CI Starts - default
    -1.0
65 volatile fix23 *cr_start_ptr; // Value at which CR Starts - default
    -2.0
66 volatile fix23 *ci_end_ptr; // Value at which CI Ends - default 1.0
67 volatile fix23 *cr_end_ptr; // Value at which CR Ends - default 1.0
68
69 volatile uint32_t *final_count_ptr;
70 volatile int32_t *cursor_posx_ptr;
71 volatile int32_t *cursor_posy_ptr;
72 volatile uint8_t *reset_hps_ptr;
73
74 int fd;
75
76 volatile float cr_range = 3.0;
77 volatile float ci_range = 2.0;
78 volatile float cr_start = -2.0;
79 volatile float ci_start = -1.0;
80
81 volatile int cursor_x = 0;
82 volatile int cursor_y = 0;
83
84 // from lab 1
85 /*
    *****
86 * Subroutine to send a string of text to the VGA monitor
87 *****
    */

```



```

88 void VGA_text(int x, int y, char * text_ptr)
89 {
90     volatile char * character_buffer = (char *) vga_char_ptr ;    //
91         VGA character buffer
92     int offset;
93     /* assume that the text string fits on one line */
94     offset = (y << 7) + x;
95     while ( *(text_ptr) )
96     {
97         // write to the character buffer
98         *(character_buffer + offset) = *(text_ptr);
99         ++text_ptr;
100         ++offset;
101     }
102 }
103 /*
104     *****
105     * Subroutine to clear text to the VGA monitor
106     *****
107     */
108 void VGA_text_clear()
109 {
110     volatile char * character_buffer = (char *) vga_char_ptr ;    //
111         VGA character buffer
112     int offset, x, y;
113     for (x=0; x<79; x++){
114         for (y=0; y<59; y++){
115             /* assume that the text string fits on one line */
116             offset = (y << 7) + x;
117             // write to the character buffer
118             *(character_buffer + offset) = '\0';
119         }
120     }
121 }
122
123 int clamp(int x, int lo, int hi) {
124     if (x < lo) return lo;
125     if (x > hi) return hi;
126     return x;
127 }
128
129 // Left click zoom IN by 2x
130 // Right click zoom OUT by 2x
131 // Code from 5760 website

```

```

130 void *mouse_thread(void *unused)
131 {
132     int mouse_fd, bytes;
133     unsigned char data[3];
134
135     const char *pDevice = "/dev/input/mice";
136
137     // Open Mouse
138     mouse_fd = open(pDevice, O_RDWR);
139     if (mouse_fd == -1)
140     {
141         printf("ERROR_Opening_%s\n", pDevice);
142         return NULL;
143     }
144
145     int left, right, middle;
146     signed char x, y;
147
148     // this will track previous button states to detect press edges
149     int prev_left = 0;
150     int prev_right = 0;
151     int mouse_x = 0;
152     int mouse_y = 0;
153
154     while (1)
155     {
156         // Read Mouse
157         bytes = read(mouse_fd, data, sizeof(data));
158
159         cursor_x = mouse_x + 320;
160         cursor_y = mouse_y + 240;
161
162         if (bytes > 0)
163         {
164             left = data[0] & 0x1;
165             right = data[0] & 0x2;
166             middle = data[0] & 0x4;
167
168             x = data[1];
169             y = data[2];
170
171             mouse_x = clamp(mouse_x + x, -320, 320);
172             mouse_y = clamp(mouse_y - y, -240, 240);
173
174             // printf(
175             //     "mouse x: %d, mouse y: %d\n",
176             //     mouse_x,

```

```

177         //      mouse_y
178         // );
179
180         // Fractional position of cursor on screen (0.0 to 1.0)
181         float frac_x = (mouse_x + 320.0f) / 640.0f;
182         float frac_y = (mouse_y + 240.0f) / 480.0f;
183
184         // Complex coordinate under the cursor
185         float cr_mouse = cr_start + frac_x * cr_range;
186         float ci_mouse = ci_start + frac_y * ci_range;
187
188         // printf(
189         //      "complex under cursor: cr=%f, ci=%f\n",
190         //      cr_mouse,
191         //      ci_mouse
192         // );
193
194         // use this for debugging
195         // printf("x=%d, y=%d, left=%d, middle=%d, right=%d\n",
196         //      x, y, left, middle, right);
197         ##### BEGIN AI-GENERATED CODE
198         #####
199
200         if ((left && !prev_left) || (right && !prev_right))
201         {
202             if (left && !prev_left)
203             {
204                 // Zoom IN by 2x
205                 cr_range *= 0.5f;
206                 ci_range *= 0.5f;
207             }
208             else
209             {
210                 // Zoom OUT by 2x
211                 cr_range *= 2.0f;
212                 ci_range *= 2.0f;
213             }
214
215             // Recompute start so the point under the cursor
216             // stays fixed
217             cr_start = cr_mouse - frac_x * cr_range;
218             ci_start = ci_mouse - frac_y * ci_range;
219         }
220
221         prev_left  = left;
222         prev_right = right;

```

```

221     }
222         ##### END AI-GENERATED CODE
223         #####
224
225     close(mouse_fd);
226     return NULL;
227 }
228
229 int main()
230 {
231     fputs("\x1b[2J\x1b[H", stdout);
232
233     if( ( fd = open( "/dev/mem", ( O_RDWR | O_SYNC ) ) ) == -1 )
234     {
235         printf( "ERROR: could not open \"/dev/mem\"...\n" );
236         return( 1 );
237     }
238
239     h2p_lw_virtual_base = mmap( NULL, HW_REGS_SPAN, ( PROT_READ |
240     PROT_WRITE ), MAP_SHARED, fd, HW_REGS_BASE );
241     if( h2p_lw_virtual_base == MAP_FAILED ) {
242         printf( "ERROR: mmap1() failed...\n" );
243         close( fd );
244         return(1);
245     }
246
247     // === get VGA char addr =====
248     // get virtual addr that maps to physical
249     // vga_char_virtual_base = mmap( NULL, FPGA_CHAR_SPAN, (
250     PROT_READ | PROT_WRITE ), MAP_SHARED, fd, FPGA_CHAR_BASE );
251     // if( vga_char_virtual_base == MAP_FAILED ) {
252     //     printf( "ERROR: mmap2() failed...\n" );
253     //     close( fd );
254     //     return(1);
255     // }
256
257     // Get the address that maps to the FPGA LED control
258     // vga_char_ptr = (unsigned int *) (vga_char_virtual_base);
259
260     cr_step_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
261     CR_STEP_OFFSET);
262     cr_iter_step_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
263     CR_ITER_STEP_OFFSET);
264     ci_iter_step_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
265     CI_ITER_STEP_OFFSET);

```

```

260 cr_start_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
    CR_START_OFFSET);
261 ci_start_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
    CI_START_OFFSET);
262 cr_end_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
    CR_END_OFFSET);
263 ci_end_ptr = (volatile fix23 *) (h2p_lw_virtual_base +
    CI_END_OFFSET);
264
265 final_count_ptr = (volatile uint32_t *) (h2p_lw_virtual_base +
    FINAL_COUNT_OFFSET);
266 cursor_posx_ptr = (volatile int32_t *) (h2p_lw_virtual_base +
    CURSOR_POSX_OFFSET);
267 cursor_posy_ptr = (volatile int32_t *) (h2p_lw_virtual_base +
    CURSOR_POSY_OFFSET);
268 reset_hps_ptr = (volatile uint8_t *) (h2p_lw_virtual_base +
    HPS_RESET_OFFSET);
269
270
271 // clear the text
272 // VGA_text_clear();
273
274 // created thread for mouse
275 pthread_t thread_mouse;
276 pthread_create(&thread_mouse, NULL, *mouse_thread, NULL);
277
278
279
280 char time_str[40];
281
282 for(;;)
283 {
284
285     if (*reset_hps_ptr)
286     {
287         cr_range = 3.0;
288         ci_range = 2.0;
289         cr_start = -2.0;
290         ci_start = -1.0;
291
292         cursor_x = 0;
293         cursor_y = 0;
294     }
295
296     *cr_step_ptr = float2fix((cr_range / 640.0));
297     *cr_iter_step_ptr = float2fix((cr_range / 640.0) *
    NUM_ITERATORS);

```

```

298     *ci_iter_step_ptr = float2fix((ci_range / 480.0));
299
300     *cr_start_ptr = float2fix(cr_start);
301     *ci_start_ptr = float2fix(ci_start);
302     *cr_end_ptr = float2fix((cr_start + cr_range));
303     *ci_end_ptr = float2fix((ci_start + ci_range));
304
305     *cursor_posx_ptr = cursor_x;
306     *cursor_posy_ptr = cursor_y;
307
308     printf("\033[H");
309
310     printf("Rendering took %f ms\033[K\n", 1000.0 * ((float)*
        final_count_ptr) / (CLK_SPEED));
311     printf("Cursor position x: %d, y: %d\033[K\n", cursor_x,
        cursor_y);
312     printf("Corners UL: (%f, %f) UR: (%f, %f) BL: (%f, %f) BR:
        (%f, %f)\033[K\n", cr_start, ci_start,
313
314
315

```

```

316
317         // added this code to print on the VGA (check this)
318         // float render_time = 1000.0 * ((float)*final_count_ptr) /
           CLK_SPEED;
319         // sprintf(time_str, "Render: %.2f ms    ", render_time);
320         // VGA_text(0, 0, time_str);
321     }
322
323     // thread for mouse
324     pthread_join(thread_mouse, NULL);
325
326 }

```

Listing 1: My C Function

```

1
2 `timescale 1ns/1ns
3
4 module testbench_mandelbrot();
5     localparam integer SCALE = 8388608; // 2^23, needed for using 4:23
           Fixed Point
6     reg clk_50, clk_25, reset;
7
8     reg signed [26:0] c_r;
9     reg signed [26:0] c_i;
10
11     wire signed [26:0] z_new;
12     wire [9:0] result;
13
14     localparam cr_float = 1.5;
15     localparam ci_float = -0.75;
16
17     // Instantiate Device Under Test
18     mandlebrot_system DUT (
19         .clk(clk_50),
20         .reset(reset),
21
22         .c_r(c_r),
23         .c_i(c_i),
24         .z_new(z_new),
25         .result(result)
26     );

```

```

27
28 // Initialize Parameters
29 initial begin
30     clk_50 = 1'b0;
31     clk_25 = 1'b0;
32
33     c_r = cr_float * SCALE;
34     c_i = ci_float * SCALE;
35 end
36
37 // Dump waveforms and stop simulation
38 initial begin
39     $dumpfile("mandelbrot.vcd");
40     $dumpvars(0, testbench_mandelbrot);
41     #100000;
42     $display("Result: %d, Escaped: %b, Ready: %b", result, DUT.
43             escaped, DUT.ready);
44     $finish;
45 end
46
47 // Reset system
48 initial begin
49     reset = 1'b0;
50     #10
51     reset = 1'b1;
52     #30
53     reset = 1'b0;
54 end
55
56 //Toggle the clocks
57 always begin
58     #10
59     clk_50 = !clk_50;
60 end
61
62 always begin
63     #20
64     clk_25 = !clk_25;
65 end
66
67 endmodule
68
69 //////////////////////////////////////
70 //// signed mult of 4.23 format 2'comp////////////////////////////////////
71 //////////////////////////////////////
72 module signed_mult (out, a, b);

```



```

73     output    signed    [26:0]    out;
74     input     signed    [26:0]    a;
75     input     signed    [26:0]    b;
76     // intermediate full bit length
77     wire      signed    [53:0]    mult_out;
78     assign mult_out = a * b;
79     // select bits for 4.23 fixed point
80     assign out = mult_out[49:23];
81 endmodule
82 //////////////////////////////////////
83
84
85 module mandelbrot_system#(
86     parameter integer MAX_ITERATIONS = 1000
87 )(
88     input wire clk,
89     input wire reset,
90
91     input wire signed [26:0] c_r,
92     input wire signed [26:0] c_i,
93
94     output wire signed [26:0] z_new,
95     output reg          escaped,
96     output reg          ready,
97     output reg          [$clog2(MAX_ITERATIONS)-1:0] counter,
98     output reg          [$clog2(MAX_ITERATIONS)-1:0] result
99 );
100
101     localparam integer SCALE = 8388608; // 2^23, needed for using
102         4:23 Fixed Point
103     // maybe make these first two inputs
104     localparam ESCAPE_BORDER = 4 * SCALE;
105
106     // Primary Bulb Rectangle Edges
107     localparam MAIN_RECT_LEFT    = -5062945;
108     localparam MAIN_RECT_RIGHT   = 86864;
109     localparam MAIN_RECT_TOP     = 2965792;
110     localparam MAIN_RECT_BOTTOM  = -2965793;
111
112     // Secondary Bulb Rectangle Edges
113     localparam SEC_RECT_LEFT     = -9871463;
114     localparam SEC_RECT_RIGHT    = -6905670;
115     localparam SEC_RECT_BOTTOM   = -1482855;
116     localparam SEC_RECT_TOP      = 1482854;
117
118     reg signed [26:0] zr;

```

```

119     reg signed [26:0] zi;
120     reg signed [26:0] zr_sqd;
121     reg signed [26:0] zi_sqd;
122
123
124     wire signed [26:0] zr_new;
125     wire signed [26:0] zi_new;
126     wire signed [26:0] zr_sqd_new;
127     wire signed [26:0] zi_sqd_new;
128
129
130     always @(posedge clk) begin
131         if (reset) begin
132             zr <= 0;
133             zi <= 0;
134             zr_sqd <= 0; // may soon just make these equal to zero
135             as these initial comnditons wont change
136             zi_sqd <= 0;
137             counter <= 1;
138             escaped <= 0;
139             ready <= 0;
140             result <= 0;
141         end
142         else if ((c_r >= MAIN_RECT_LEFT && c_r <= MAIN_RECT_RIGHT &&
143             c_i >= MAIN_RECT_BOTTOM && c_i <= MAIN_RECT_TOP)
144             || (c_r >= SEC_RECT_LEFT && c_r <=
145                 SEC_RECT_RIGHT && c_i >= SEC_RECT_BOTTOM
146                 && c_i <= SEC_RECT_TOP) ) begin
147             result <= MAX_ITERATIONS;
148             ready <= 1;
149         end
150         else if (!escaped && (counter < MAX_ITERATIONS) && (zr > -2*
151             SCALE) && (zr < 2*SCALE) && (zi > -2*SCALE) && (zi < 2*
152             SCALE)) begin
153             zr <= zr_new;
154             zi <= zi_new;
155             zr_sqd <= zr_sqd_new;
156             zi_sqd <= zi_sqd_new;
157             counter <= counter + 1;
158
159             if (z_new >= ESCAPE_BORDER) begin
160                 escaped <= 1;
161                 ready <= 1;
162                 result <= counter;
163             end
164         end
165         else if ((counter < MAX_ITERATIONS) && !escaped) begin

```

```

160         escaped <= 1;
161         result <= counter;
162         ready <= 1;
163     end
164     else begin
165         ready <= 1;
166     end
167 end
168
169
170 wire signed [26:0] z_n2_sum;
171 assign z_n2_sum = zr_sqd - zi_sqd;
172 assign zr_new = c_r + z_n2_sum;
173
174 signed_mult zr_new_squared
175 (
176     .out(zr_sqd_new),
177     .a(zr_new),
178     .b(zr_new)
179 );
180
181 wire signed [26:0] zri_n_prod;
182
183 signed_mult zri_n_product
184 (
185     .out(zri_n_prod),
186     .a(zr),
187     .b(zi)
188 );
189
190 wire signed [26:0] zri_n_prod_div;
191 assign zri_n_prod_div = zri_n_prod <<< 1;
192
193 assign zi_new = zri_n_prod_div + c_i;
194
195 signed_mult zri_new_squared
196 (
197     .out(zi_sqd_new),
198     .a(zi_new),
199     .b(zi_new)
200 );
201
202 assign z_new = zr_sqd_new + zi_sqd_new;
203
204
205 endmodule

```

Listing 2: mandelbrot_ierator.v

```

1 module DE1_SoC_Computer (
2     //////////////////////////////////////////
3     // FPGA Pins
4     //////////////////////////////////////////
5
6     // Clock pins
7     CLOCK_50 ,
8     CLOCK2_50 ,
9     CLOCK3_50 ,
10    CLOCK4_50 ,
11
12    // ADC
13    ADC_CS_N ,
14    ADC_DIN ,
15    ADC_DOUT ,
16    ADC_SCLK ,
17
18    // Audio
19    AUD_ADCDAT ,
20    AUD_ADCLRCK ,
21    AUD_BCLK ,
22    AUD_DACDAT ,
23    AUD_DACLCK ,
24    AUD_XCK ,
25
26    // SDRAM
27    DRAM_ADDR ,
28    DRAM_BA ,
29    DRAM_CAS_N ,
30    DRAM_CKE ,
31    DRAM_CLK ,
32    DRAM_CS_N ,
33    DRAM_DQ ,
34    DRAM_LDQM ,
35    DRAM_RAS_N ,
36    DRAM_UDQM ,
37    DRAM_WE_N ,
38
39    // I2C Bus for Configuration of the Audio and Video-In Chips
40    FPGA_I2C_SCLK ,
41    FPGA_I2C_SDAT ,
42
43    // 40-Pin Headers
44    GPIO_0 ,
45    GPIO_1 ,
46
47    // Seven Segment Displays

```

```

48  HEX0 ,
49  HEX1 ,
50  HEX2 ,
51  HEX3 ,
52  HEX4 ,
53  HEX5 ,
54
55  // IR
56  IRDA_RXD ,
57  IRDA_TXD ,
58
59  // Pushbuttons
60  KEY ,
61
62  // LEDs
63  LEDR ,
64
65  // PS2 Ports
66  PS2_CLK ,
67  PS2_DAT ,
68
69  PS2_CLK2 ,
70  PS2_DAT2 ,
71
72  // Slider Switches
73  SW ,
74
75  // Video-In
76  TD_CLK27 ,
77  TD_DATA ,
78  TD_HS ,
79  TD_RESET_N ,
80  TD_VS ,
81
82  // VGA
83  VGA_B ,
84  VGA_BLANK_N ,
85  VGA_CLK ,
86  VGA_G ,
87  VGA_HS ,
88  VGA_R ,
89  VGA_SYNC_N ,
90  VGA_VS ,
91
92  //////////////////////////////////////
93  // HPS Pins
94  //////////////////////////////////////

```

```

95
96 // DDR3 SDRAM
97 HPS_DDR3_ADDR ,
98 HPS_DDR3_BA ,
99 HPS_DDR3_CAS_N ,
100 HPS_DDR3_CKE ,
101 HPS_DDR3_CK_N ,
102 HPS_DDR3_CK_P ,
103 HPS_DDR3_CS_N ,
104 HPS_DDR3_DM ,
105 HPS_DDR3_DQ ,
106 HPS_DDR3_DQS_N ,
107 HPS_DDR3_DQS_P ,
108 HPS_DDR3_ODT ,
109 HPS_DDR3_RAS_N ,
110 HPS_DDR3_RESET_N ,
111 HPS_DDR3_RZQ ,
112 HPS_DDR3_WE_N ,
113
114 // Ethernet
115 HPS_ENET_GTX_CLK ,
116 HPS_ENET_INT_N ,
117 HPS_ENET_MDC ,
118 HPS_ENET_MDIO ,
119 HPS_ENET_RX_CLK ,
120 HPS_ENET_RX_DATA ,
121 HPS_ENET_RX_DV ,
122 HPS_ENET_TX_DATA ,
123 HPS_ENET_TX_EN ,
124
125 // Flash
126 HPS_FLASH_DATA ,
127 HPS_FLASH_DCLK ,
128 HPS_FLASH_NCS0 ,
129
130 // Accelerometer
131 HPS_GSENSOR_INT ,
132
133 // General Purpose I/O
134 HPS_GPIO ,
135
136 // I2C
137 HPS_I2C_CONTROL ,
138 HPS_I2C1_SCLK ,
139 HPS_I2C1_SDAT ,
140 HPS_I2C2_SCLK ,
141 HPS_I2C2_SDAT ,

```

```

142
143 // Pushbutton
144 HPS_KEY ,
145
146 // LED
147 HPS_LED ,
148
149 // SD Card
150 HPS_SD_CLK ,
151 HPS_SD_CMD ,
152 HPS_SD_DATA ,
153
154 // SPI
155 HPS_SPIM_CLK ,
156 HPS_SPIM_MISO ,
157 HPS_SPIM_MOSI ,
158 HPS_SPIM_SS ,
159
160 // UART
161 HPS_UART_RX ,
162 HPS_UART_TX ,
163
164 // USB
165 HPS_CONV_USB_N ,
166 HPS_USB_CLKOUT ,
167 HPS_USB_DATA ,
168 HPS_USB_DIR ,
169 HPS_USB_NXT ,
170 HPS_USB_STP
171 );
172
173 //=====
174 // PARAMETER declarations
175 //=====
176
177
178 //=====
179 // PORT declarations
180 //=====
181
182 //////////////////////////////////////
183 // FPGA Pins
184 //////////////////////////////////////
185
186 // Clock pins
187 input          CLOCK_50;
188 input          CLOCK2_50;

```

```

189 input          CLOCK3_50;
190 input          CLOCK4_50;
191
192 // ADC
193 inout          ADC_CS_N;
194 output          ADC_DIN;
195 input          ADC_DOUT;
196 output          ADC_SCLK;
197
198 // Audio
199 input          AUD_ADCDAT;
200 inout          AUD_ADCLRCK;
201 inout          AUD_BCLK;
202 output          AUD_DACDAT;
203 inout          AUD_DACLCK;
204 output          AUD_XCK;
205
206 // SDRAM
207 output          [12: 0] DRAM_ADDR;
208 output          [ 1: 0] DRAM_BA;
209 output          DRAM_CAS_N;
210 output          DRAM_CKE;
211 output          DRAM_CLK;
212 output          DRAM_CS_N;
213 inout          [15: 0] DRAM_DQ;
214 output          DRAM_LDQM;
215 output          DRAM_RAS_N;
216 output          DRAM_UDQM;
217 output          DRAM_WE_N;
218
219 // I2C Bus for Configuration of the Audio and Video-In Chips
220 output          FPGA_I2C_SCLK;
221 inout          FPGA_I2C_SDAT;
222
223 // 40-pin headers
224 inout          [35: 0] GPIO_0;
225 inout          [35: 0] GPIO_1;
226
227 // Seven Segment Displays
228 output          [ 6: 0] HEX0;
229 output          [ 6: 0] HEX1;
230 output          [ 6: 0] HEX2;
231 output          [ 6: 0] HEX3;
232 output          [ 6: 0] HEX4;
233 output          [ 6: 0] HEX5;
234
235 // IR

```



```

236 input                IRDA_RXD;
237 output               IRDA_TXD;
238
239 // Pushbuttons
240 input                 [ 3: 0] KEY;
241
242 // LEDs
243 output               [ 9: 0] LEDR;
244
245 // PS2 Ports
246 inout               PS2_CLK;
247 inout               PS2_DAT;
248
249 inout               PS2_CLK2;
250 inout               PS2_DAT2;
251
252 // Slider Switches
253 input                 [ 9: 0] SW;
254
255 // Video-In
256 input               TD_CLK27;
257 input                 [ 7: 0] TD_DATA;
258 input               TD_HS;
259 output              TD_RESET_N;
260 input               TD_VS;
261
262 // VGA
263 output                 [ 7: 0] VGA_B;
264 output               VGA_BLANK_N;
265 output               VGA_CLK;
266 output                 [ 7: 0] VGA_G;
267 output               VGA_HS;
268 output                 [ 7: 0] VGA_R;
269 output               VGA_SYNC_N;
270 output               VGA_VS;
271
272
273
274 //////////////////////////////////////
275 // HPS Pins
276 //////////////////////////////////////
277
278 // DDR3 SDRAM
279 output                 [14: 0] HPS_DDR3_ADDR;
280 output                 [ 2: 0] HPS_DDR3_BA;
281 output               HPS_DDR3_CAS_N;
282 output               HPS_DDR3_CKE;

```

```

283 output          HPS_DDR3_CK_N;
284 output          HPS_DDR3_CK_P;
285 output          HPS_DDR3_CS_N;
286 output          [ 3: 0] HPS_DDR3_DM;
287 inout           [31: 0] HPS_DDR3_DQ;
288 inout           [ 3: 0] HPS_DDR3_DQS_N;
289 inout           [ 3: 0] HPS_DDR3_DQS_P;
290 output          HPS_DDR3_ODT;
291 output          HPS_DDR3_RAS_N;
292 output          HPS_DDR3_RESET_N;
293 input           HPS_DDR3_RZQ;
294 output          HPS_DDR3_WE_N;
295
296 // Ethernet
297 output          HPS_ENET_GTX_CLK;
298 inout           HPS_ENET_INT_N;
299 output          HPS_ENET_MDC;
300 inout           HPS_ENET_MDIO;
301 input           HPS_ENET_RX_CLK;
302 input           [ 3: 0] HPS_ENET_RX_DATA;
303 input           HPS_ENET_RX_DV;
304 output          [ 3: 0] HPS_ENET_TX_DATA;
305 output          HPS_ENET_TX_EN;
306
307 // Flash
308 inout           [ 3: 0] HPS_FLASH_DATA;
309 output          HPS_FLASH_DCLK;
310 output          HPS_FLASH_NCS0;
311
312 // Accelerometer
313 inout           HPS_GSENSOR_INT;
314
315 // General Purpose I/O
316 inout           [ 1: 0] HPS_GPIO;
317
318 // I2C
319 inout           HPS_I2C_CONTROL;
320 inout           HPS_I2C1_SCLK;
321 inout           HPS_I2C1_SDAT;
322 inout           HPS_I2C2_SCLK;
323 inout           HPS_I2C2_SDAT;
324
325 // Pushbutton
326 inout           HPS_KEY;
327
328 // LED
329 inout           HPS_LED;

```

```

330
331 // SD Card
332 output          HPS_SD_CLK;
333 inout           HPS_SD_CMD;
334 inout           [ 3: 0] HPS_SD_DATA;
335
336 // SPI
337 output          HPS_SPIM_CLK;
338 input           HPS_SPIM_MISO;
339 output          HPS_SPIM_MOSI;
340 inout           HPS_SPIM_SS;
341
342 // UART
343 input           HPS_UART_RX;
344 output          HPS_UART_TX;
345
346 // USB
347 inout           HPS_CONV_USB_N;
348 input           HPS_USB_CLKOUT;
349 inout           [ 7: 0] HPS_USB_DATA;
350 input           HPS_USB_DIR;
351 input           HPS_USB_NXT;
352 output          HPS_USB_STP;
353
354 //=====
355 // Parameter declarations
356 //=====
357
358 localparam integer NUM_ITERATORS = 87; // Number of iterators to
    instantiate / MUST MATCH C MACRO
359 localparam integer SCALE = 8388608; // 2^23 for fixed to
    floating conversion
360 localparam integer WIDTH = 640; // X width
361 localparam integer HEIGHT = 480; // Y height
362 localparam integer MEMROW_LENGTH = (WIDTH + NUM_ITERATORS - 1) /
    NUM_ITERATORS; // ceil(WIDTH/N)
363 localparam integer DEPTH = MEMROW_LENGTH * HEIGHT;
364 localparam integer ADDR_W = 19; // enough for up to 307200
365 localparam integer MAX_ITERATIONS = 1000;
366
367 //=====
368 // REG/WIRE declarations
369 //=====
370
371 wire            [15: 0] hex3_hex0;
372 assign HEX4 = 7'b1111111;
373 assign HEX5 = 7'b1111111;

```

```

374
375 HexDigit Digit0(HEX0, hex3_hex0[3:0]);
376 HexDigit Digit1(HEX1, hex3_hex0[7:4]);
377 HexDigit Digit2(HEX2, hex3_hex0[11:8]);
378 HexDigit Digit3(HEX3, hex3_hex0[15:12]);
379
380 // VGA clock and reset lines
381 wire vga_pll_lock ;
382 wire vga_pll ;
383 reg vga_reset ;
384
385 // M10k memory control and data
386 wire [7:0] M10k_out[NUM_ITERATORS];
387 reg [7:0] write_data[NUM_ITERATORS] ;
388 reg [18:0] write_address[NUM_ITERATORS] ;
389 wire [ADDR_W - 1:0] read_address ;
390 reg write_enable[NUM_ITERATORS] ;
391
392 // M10k memory clock
393 wire M10k_pll ;
394 wire M10k_pll_locked ;
395
396 // Wires for connecting VGA driver to memory
397 wire [9:0] next_x;
398 wire [9:0] next_y;
399
400 // muxed color in for vga
401 wire [7:0] color_in;
402 wire [7:0] bank_select;
403
404 //=====
405 // Global always block - handle VGA reset signal
406 //=====
407
408 reg [7:0] reset_hps;
409
410 always@(posedge M10k_pll) begin
411     if (~KEY[0]) begin
412         vga_reset <= 1'b1 ;
413         reset_hps <= 1'b1 ;
414     end
415     else begin
416         vga_reset <= 1'b0 ;
417         reset_hps <= 1'b0 ;
418     end
419 end
420

```

```

421 assign bank_select = next_x % NUM_ITERATORS;
422 assign color_in = M10k_out[bank_select];
423
424 // Per iterator mandelbrot parameters
425 reg signed [26:0] cr_in[NUM_ITERATORS];           // current cr
    being computed
426 reg signed [26:0] ci_in[NUM_ITERATORS];           // current ci
    being computed
427 reg [9:0] x_coord[NUM_ITERATORS];                 // x_coord of
    pixel being computed
428 reg [9:0] y_coord[NUM_ITERATORS];                 // y_coord of
    pixel being computed
429 reg [18:0] write_addr_x[NUM_ITERATORS];
430 reg [18:0] write_addr_y[NUM_ITERATORS];
431 wire [26:0] z_new[NUM_ITERATORS];                 // z output of
    iterator (not used externally)
432 wire [9:0] result[NUM_ITERATORS];                 // how many
    cycles before it escaped, or 0 if it did not
433 wire signed [26:0] cr_reset_val[NUM_ITERATORS];   // cr value of
    the first pixel this iterator computes
434 reg mandel_reset[NUM_ITERATORS];                 // reset signal
    for new pixel
435 wire escaped[NUM_ITERATORS];                       // did this
    pixel escape?
436 wire ready[NUM_ITERATORS];                       // goes high
    when computation on pixel is done
437
438 reg [NUM_ITERATORS - 1:0] done; // bitwise done signal
439 wire [NUM_ITERATORS - 1:0] done_condition; // all ones, and
    with done to check if all iterators have finished
440 wire [31:0] c_count;
441
442 assign done_condition = {NUM_ITERATORS{1'b1}};
443
444 // Initial params from HPS
445 wire signed [26:0] cr_step;                        // cr_little_step *
    NUM_ITERATORS
446 wire signed [26:0] ci_step;                        // ci_little_step *
    NUM_ITERATORS
447 wire signed [26:0] cr_start;                       // cr low end of range
448 wire signed [26:0] cr_stop;                       // cr high end of range
449 wire signed [26:0] ci_start;                       // ci low end of range
450 wire signed [26:0] ci_stop;                       // ci high end of range
451 wire signed [26:0] cr_little_step; // cr step per pixel (x)
452
453 wire [9:0] cursor_x;
454 wire [9:0] cursor_y;

```

```

455
456 wire [26:0] temp;
457
458 assign read_address = (next_y * MEMROW_LENGTH) + (next_x /
    NUM_ITERATORS);
459
460 //=====
461 // Accumulate initial condition for cr for each iterator
462 // Avoids expensive multiplies
463 //=====
464 genvar j;
465 generate
466     begin : CR_RESET_BASE
467         assign cr_reset_val[0] = cr_start;
468     end
469
470     for (j = 1; j < NUM_ITERATORS; j = j + 1) begin : CR_RESET_ACC
471         assign cr_reset_val[j] = cr_reset_val[j - 1] +
            cr_little_step;
472     end
473
474 endgenerate
475
476 genvar i;
477 generate
478     for (i=0; i < NUM_ITERATORS; i = i + 1) begin: iterator_instance
479
480         /* Mandelbrot iterator */
481         mandel_1dsp mandel(
482             .clk                (M10k_pll),
483             .reset               (mandel_reset[i]),
484             .c_r                 (cr_in[i]),
485             .c_i                 (ci_in[i]),
486             .z_new               (z_new[i]),
487             .escaped             (escaped[i]),
488             .ready               (ready[i]),
489             .counter             (temp),
490             .result              (result[i])
491         );
492
493         /* M10K block per iterator*/
494         M10K_1000_8#(DEPTH) pixel_data(
495
496             .q(M10k_out[i]), // contains
497                 pixel color (8 bit) for
498                 display
499             .d(write_data[i]),
500             .write_address(write_address

```

```

498                                     [i]),
499                                     .read_address(read_address),
500                                     .we(write_enable[i]),
501                                     .clk(M10k_pll)
502     );
503
504     // Write address per iterator
505     // assign write_address[i] = (y_coord[i] * MEMROW_LENGTH) + (
506     // x_coord[i] / NUM_ITERATORS);
507
508     always@(posedge M10k_pll) begin
509         if (~KEY[0]) begin
510
511             cr_in[i] <= cr_reset_val[i];
512             ci_in[i] <= ci_start;
513
514             done[i] <= 1'b0;
515
516             mandel_reset[i] <= 1'b1;
517             write_enable[i] <= 1'b0;
518             x_coord[i] <= i;
519             y_coord[i] <= 0;
520             write_addr_x[i] <= 0;
521             write_addr_y[i] <= 0;
522
523         end
524         else begin
525             mandel_reset[i] <= 1'b0; // assert reset low
526             write_enable[i] <= 1'b0;
527
528             if (ready[i] & ~mandel_reset[i]) begin
529
530                 write_address[i] <= write_addr_x[i] +
531                     write_addr_y[i];
532
533                 // write_data[i] <= escaped[i] ? 8'b0000_0000 : 8'
534                 // b1111_1111 ;
535
536                 if ((x_coord[i] == cursor_x && y_coord[i] >=
537                     cursor_y - 5 && y_coord[i] <= cursor_y + 5)
538                     ||
539                     (y_coord[i] == cursor_y && x_coord[i] >=
540                     cursor_x - 5 && x_coord[i] <= cursor_x +
541                     5)) begin
542                     write_data[i] <= 8'b_010_111_11;
543                 end
544                 else if (!escaped[i]) begin
545                     write_data[i] <= 8'b_000_000_00 ; // black

```

```

537     end
538     else if (result[i] >= (MAX_ITERATIONS >>> 1))
539         begin
540             write_data[i] <= 8'b_011_001_00 ; // white
541         end
542     else if (result[i] >= (MAX_ITERATIONS >>> 2))
543         begin
544             write_data[i] <= 8'b_011_001_00 ;
545         end
546     else if (result[i] >= (MAX_ITERATIONS >>> 3))
547         begin
548             write_data[i] <= 8'b_101_010_01 ;
549         end
550     else if (result[i] >= (MAX_ITERATIONS >>> 4))
551         begin
552             write_data[i] <= 8'b_011_001_01 ;
553         end
554     else if (result[i] >= (MAX_ITERATIONS >>> 5))
555         begin
556             write_data[i] <= 8'b_001_001_01 ;
557         end
558     else if (result[i] >= (MAX_ITERATIONS >>> 6))
559         begin
560             write_data[i] <= 8'b_011_010_10 ;
561         end
562     else if (result[i] >= (MAX_ITERATIONS >>> 7))
563         begin
564             write_data[i] <= 8'b_010_100_10 ;
565         end
566     else if (result[i] >= (MAX_ITERATIONS >>> 8))
567         begin
568             write_data[i] <= 8'b_010_100_10 ;
569         end
570     else begin
571         write_data[i] <= 8'b_010_100_10 ;
572     end
573
574     write_enable[i] <= 1;
575
576     if ((x_coord[i] + NUM_ITERATORS) > 639) begin
577         if ((y_coord[i] + 1) > 479) begin
578             done[i] <= 1'b1;
579             ci_in[i] <= ci_start;
580             cr_in[i] <= cr_reset_val[i];
581
582             x_coord[i] <= i;
583             y_coord[i] <= 0;

```



```

576
577         write_addr_x[i] <= 0;
578         write_addr_y[i] <= 0;
579
580     end
581     else begin
582         ci_in[i] <= ci_in[i] + ci_step;
583         cr_in[i] <= cr_reset_val[i];
584
585         x_coord[i] <= i;
586         y_coord[i] <= y_coord[i] + 1;
587
588         write_addr_x[i] <= 0;
589         write_addr_y[i] <= write_addr_y[i] +
            MEMROW_LENGTH;
590     end
591 end
592 else begin
593     cr_in[i] <= cr_in[i] + cr_step;
594     ci_in[i] <= ci_in[i];
595
596     x_coord[i] <= x_coord[i] + NUM_ITERATORS;
597     y_coord[i] <= y_coord[i];
598
599     write_addr_x[i] <= write_addr_x[i] + 1;
600     write_addr_y[i] <= write_addr_y[i];
601 end
602
603 // toggle reset
604 mandel_reset[i] <= 1'b1;
605
606     end
607
608     end
609
610     end
611
612     end
613 endgenerate
614
615 // Instantiate counter for timing
616 counter c
617 (
618     .clk(M10k_pll),
619     .rst(~KEY[0]),
620     .stop(done == done_condition),
621     .count(c_count)

```

```

622 );
623
624 // Instantiate VGA driver
625 vga_driver DUT ( .clock(vga_pll),
626                  .reset(vga_reset),
627                  .color_in(color_in),    // Pixel color
628                  .next_x(next_x),        // This (and
629                  .next_y(next_y),        // This (and
630                  .hsync(VGA_HS),
631                  .vsync(VGA_VS),
632                  .red(VGA_R),
633                  .green(VGA_G),
634                  .blue(VGA_B),
635                  .sync(VGA_SYNC_N),
636                  .clk(VGA_CLK),
637                  .blank(VGA_BLANK_N)
638 );
639
640
641 //=====
642 // Structural coding
643 //=====
644 // From Qsys
645
646
647 Computer_System The_System (
648     //////////////////////////////////////
649     // FPGA Side
650     //////////////////////////////////////
651     .vga_pio_locked_export      (vga_pll_lock),          //
652     .vga_pio_locked_export      vga_pio_locked.export    //
653     .vga_pio_outclk0_clk        (vga_pll),                //
654     .vga_pio_outclk0_clk        vga_pio_outclk0.clk
655     .m10k_pll_locked_export     (M10k_pll_locked),        //
656     .m10k_pll_locked_export     m10k_pll_locked.export
657     .m10k_pll_outclk0_clk        (M10k_pll),              //
658     .m10k_pll_outclk0_clk        m10k_pll_outclk0.clk
659
660     // Global signals
661     .system_pll_ref_clk_clk      (CLOCK_50),
662     .system_pll_ref_reset_reset (1'b0),

```

```

660 //////////////////////////////////////////////////
661 // HPS Side
662 //////////////////////////////////////////////////
663
664
665 //////////////////////////////////////////////////
666 // PIO
667 //////////////////////////////////////////////////
668 .cr_step_export      (cr_step),
669 .cr_start_export     (cr_start),
670 .cr_stop_export      (cr_stop),
671 .ci_step_export      (ci_step),
672 .ci_start_export     (ci_start),
673 .ci_stop_export      (ci_stop),
674 .cr_little_step_export (cr_little_step),
675 .final_count_export  (c_count),
676 .cursor_x_export     (cursor_x),
677 .cursor_y_export     (cursor_y),
678 .reset_hps_export    (reset_hps),
679
680 // DDR3 SDRAM
681 .memory_mem_a        (HPS_DDR3_ADDR),
682 .memory_mem_ba       (HPS_DDR3_BA),
683 .memory_mem_ck       (HPS_DDR3_CK_P),
684 .memory_mem_ck_n     (HPS_DDR3_CK_N),
685 .memory_mem_cke      (HPS_DDR3_CKE),
686 .memory_mem_cs_n     (HPS_DDR3_CS_N),
687 .memory_mem_ras_n    (HPS_DDR3_RAS_N),
688 .memory_mem_cas_n    (HPS_DDR3_CAS_N),
689 .memory_mem_we_n     (HPS_DDR3_WE_N),
690 .memory_mem_reset_n  (HPS_DDR3_RESET_N),
691 .memory_mem_dq       (HPS_DDR3_DQ),
692 .memory_mem_dqs      (HPS_DDR3_DQS_P),
693 .memory_mem_dqs_n    (HPS_DDR3_DQS_N),
694 .memory_mem_odt      (HPS_DDR3_ODT),
695 .memory_mem_dm       (HPS_DDR3_DM),
696 .memory_oct_rzqin    (HPS_DDR3_RZQ),
697
698 // Ethernet
699 .hps_io_hps_io_gpio_inst_GPIO35    (HPS_ENET_INT_N),
700 .hps_io_hps_io_emac1_inst_TX_CLK   (HPS_ENET_GTX_CLK),
701 .hps_io_hps_io_emac1_inst_TXD0     (HPS_ENET_TX_DATA[0]),
702 .hps_io_hps_io_emac1_inst_TXD1     (HPS_ENET_TX_DATA[1]),
703 .hps_io_hps_io_emac1_inst_TXD2     (HPS_ENET_TX_DATA[2]),
704 .hps_io_hps_io_emac1_inst_TXD3     (HPS_ENET_TX_DATA[3]),
705 .hps_io_hps_io_emac1_inst_RXD0     (HPS_ENET_RX_DATA[0]),
706 .hps_io_hps_io_emac1_inst_MDIO     (HPS_ENET_MDIO),

```

```

707 .hps_io_hps_io_emac1_inst_MDC          (HPS_ENET_MDC),
708 .hps_io_hps_io_emac1_inst_RX_CTL      (HPS_ENET_RX_DV),
709 .hps_io_hps_io_emac1_inst_TX_CTL      (HPS_ENET_TX_EN),
710 .hps_io_hps_io_emac1_inst_RX_CLK      (HPS_ENET_RX_CLK),
711 .hps_io_hps_io_emac1_inst_RXD1        (HPS_ENET_RX_DATA[1]),
712 .hps_io_hps_io_emac1_inst_RXD2        (HPS_ENET_RX_DATA[2]),
713 .hps_io_hps_io_emac1_inst_RXD3        (HPS_ENET_RX_DATA[3]),
714
715 // Flash
716 .hps_io_hps_io_qspi_inst_I00           (HPS_FLASH_DATA[0]),
717 .hps_io_hps_io_qspi_inst_I01           (HPS_FLASH_DATA[1]),
718 .hps_io_hps_io_qspi_inst_I02           (HPS_FLASH_DATA[2]),
719 .hps_io_hps_io_qspi_inst_I03           (HPS_FLASH_DATA[3]),
720 .hps_io_hps_io_qspi_inst_SS0           (HPS_FLASH_NCS0),
721 .hps_io_hps_io_qspi_inst_CLK           (HPS_FLASH_DCLK),
722
723 // Accelerometer
724 .hps_io_hps_io_gpio_inst_GPI061        (HPS_GSENSOR_INT),
725
726 // .adc_sclk                           (ADC_SCLK),
727 // .adc_cs_n                            (ADC_CS_N),
728 // .adc_dout                            (ADC_DOUT),
729 // .adc_din                             (ADC_DIN),
730
731 // General Purpose I/O
732 .hps_io_hps_io_gpio_inst_GPI040        (HPS_GPIO[0]),
733 .hps_io_hps_io_gpio_inst_GPI041        (HPS_GPIO[1]),
734
735 // I2C
736 .hps_io_hps_io_gpio_inst_GPI048        (HPS_I2C_CONTROL),
737 .hps_io_hps_io_i2c0_inst_SDA            (HPS_I2C1_SDAT),
738 .hps_io_hps_io_i2c0_inst_SCL            (HPS_I2C1_SCLK),
739 .hps_io_hps_io_i2c1_inst_SDA            (HPS_I2C2_SDAT),
740 .hps_io_hps_io_i2c1_inst_SCL            (HPS_I2C2_SCLK),
741
742 // Pushbutton
743 .hps_io_hps_io_gpio_inst_GPI054        (HPS_KEY),
744
745 // LED
746 .hps_io_hps_io_gpio_inst_GPI053        (HPS_LED),
747
748 // SD Card
749 .hps_io_hps_io_sdio_inst_CMD            (HPS_SD_CMD),
750 .hps_io_hps_io_sdio_inst_D0            (HPS_SD_DATA[0]),
751 .hps_io_hps_io_sdio_inst_D1            (HPS_SD_DATA[1]),
752 .hps_io_hps_io_sdio_inst_CLK            (HPS_SD_CLK),
753 .hps_io_hps_io_sdio_inst_D2            (HPS_SD_DATA[2]),

```

```

754     .hps_io_hps_io_sdio_inst_D3      (HPS_SD_DATA[3]),
755
756     // SPI
757     .hps_io_hps_io_spim1_inst_CLK    (HPS_SPIM_CLK),
758     .hps_io_hps_io_spim1_inst_MOSI   (HPS_SPIM_MOSI),
759     .hps_io_hps_io_spim1_inst_MISO   (HPS_SPIM_MISO),
760     .hps_io_hps_io_spim1_inst_SS0    (HPS_SPIM_SS),
761
762     // UART
763     .hps_io_hps_io_uart0_inst_RX     (HPS_UART_RX),
764     .hps_io_hps_io_uart0_inst_TX     (HPS_UART_TX),
765
766     // USB
767     .hps_io_hps_io_gpio_inst_GPI009  (HPS_CONV_USB_N),
768     .hps_io_hps_io_usb1_inst_D0       (HPS_USB_DATA[0]),
769     .hps_io_hps_io_usb1_inst_D1       (HPS_USB_DATA[1]),
770     .hps_io_hps_io_usb1_inst_D2       (HPS_USB_DATA[2]),
771     .hps_io_hps_io_usb1_inst_D3       (HPS_USB_DATA[3]),
772     .hps_io_hps_io_usb1_inst_D4       (HPS_USB_DATA[4]),
773     .hps_io_hps_io_usb1_inst_D5       (HPS_USB_DATA[5]),
774     .hps_io_hps_io_usb1_inst_D6       (HPS_USB_DATA[6]),
775     .hps_io_hps_io_usb1_inst_D7       (HPS_USB_DATA[7]),
776     .hps_io_hps_io_usb1_inst_CLK      (HPS_USB_CLKOUT),
777     .hps_io_hps_io_usb1_inst_STP      (HPS_USB_STP),
778     .hps_io_hps_io_usb1_inst_DIR      (HPS_USB_DIR),
779     .hps_io_hps_io_usb1_inst_NXT      (HPS_USB_NXT)
780 );
781 endmodule // end top level
782
783 module counter (
784     input wire clk,
785     input wire rst,
786     input wire stop,
787     output reg [31:0] count
788 );
789
790 always@(posedge clk) begin
791
792     if (rst) begin
793         count <= 0; // reset count to 0
794     end
795
796     else if (stop) begin
797         count <= count; // paused, do nothing
798     end
799
800     else begin

```

```

801         count <= count + 1; // if not reset and not stopped,
           count
802     end
803
804     end
805
806 endmodule
807
808 // Declaration of module, include width and signedness of each input
// output
809 module vga_driver (
810     input wire clock,
811     input wire reset,
812     input [7:0] color_in,
813     output [9:0] next_x,
814     output [9:0] next_y,
815     output wire hsync,
816     output wire vsync,
817     output [7:0] red,
818     output [7:0] green,
819     output [7:0] blue,
820     output sync,
821     output clk,
822     output blank
823 );
824
825 // Horizontal parameters (measured in clock cycles)
826 parameter [9:0] H_ACTIVE      = 10'd_639 ;
827 parameter [9:0] H_FRONT      = 10'd_15 ;
828 parameter [9:0] H_PULSE      = 10'd_95 ;
829 parameter [9:0] H_BACK       = 10'd_47 ;
830
831 // Vertical parameters (measured in lines)
832 parameter [9:0] V_ACTIVE      = 10'd_479 ;
833 parameter [9:0] V_FRONT      = 10'd_9 ;
834 parameter [9:0] V_PULSE      = 10'd_1 ;
835 parameter [9:0] V_BACK       = 10'd_32 ;
836
837 // // Horizontal parameters (measured in clock cycles)
838 // parameter [9:0] H_ACTIVE      = 10'd_9 ;
839 // parameter [9:0] H_FRONT      = 10'd_4 ;
840 // parameter [9:0] H_PULSE      = 10'd_4 ;
841 // parameter [9:0] H_BACK       = 10'd_4 ;
842 // parameter [9:0] H_TOTAL      = 10'd_799 ;
843 //
844 // // Vertical parameters (measured in lines)
845 // parameter [9:0] V_ACTIVE      = 10'd_1 ;

```

```

846 // parameter [9:0] V_FRONT      = 10'd_1 ;
847 // parameter [9:0] V_PULSE      = 10'd_1 ;
848 // parameter [9:0] V_BACK       = 10'd_1 ;
849
850 // Parameters for readability
851 parameter LOW      = 1'b_0 ;
852 parameter HIGH     = 1'b_1 ;
853
854 // States (more readable)
855 parameter [7:0] H_ACTIVE_STATE   = 8'd_0 ;
856 parameter [7:0] H_FRONT_STATE   = 8'd_1 ;
857 parameter [7:0] H_PULSE_STATE   = 8'd_2 ;
858 parameter [7:0] H_BACK_STATE    = 8'd_3 ;
859
860 parameter [7:0] V_ACTIVE_STATE   = 8'd_0 ;
861 parameter [7:0] V_FRONT_STATE   = 8'd_1 ;
862 parameter [7:0] V_PULSE_STATE   = 8'd_2 ;
863 parameter [7:0] V_BACK_STATE    = 8'd_3 ;
864
865 // Clocked registers
866 reg      hysnc_reg ;
867 reg      vsync_reg ;
868 reg [7:0] red_reg ;
869 reg [7:0] green_reg ;
870 reg [7:0] blue_reg ;
871 reg      line_done ;
872
873 // Control registers
874 reg [9:0] h_counter ;
875 reg [9:0] v_counter ;
876
877 reg [7:0] h_state ;
878 reg [7:0] v_state ;
879
880 // State machine
881 always@(posedge clock) begin
882     // At reset . . .
883     if (reset) begin
884         // Zero the counters
885         h_counter <= 10'd_0 ;
886         v_counter <= 10'd_0 ;
887         // States to ACTIVE
888         h_state <= H_ACTIVE_STATE ;
889         v_state <= V_ACTIVE_STATE ;
890         // Deassert line done
891         line_done <= LOW ;
892     end

```

893
894

895

896

897
898
899
900
901
902
903
904
905

906
907
908
909

910
911
912
913
914

915
916
917

918
919
920
921
922

923
924
925

```
else begin
    //
    ////////////////////////////////////////

    //////////////////////////////////////// HORIZONTAL
    ////////////////////////////////////////

    //
    ////////////////////////////////////////

    if (h_state == H_ACTIVE_STATE) begin
        // Iterate horizontal counter, zero at end of ACTIVE
        mode
        h_counter <= (h_counter==H_ACTIVE)?10'd_0:(h_counter
            + 10'd_1) ;
        // Set hsync
        hsync_reg <= HIGH ;
        // Deassert line done
        line_done <= LOW ;
        // State transition
        h_state <= (h_counter == H_ACTIVE)?H_FRONT_STATE:
            H_ACTIVE_STATE ;
    end
    // Assert done flag, wait here for reset
    if (h_state == H_FRONT_STATE) begin
        // Iterate horizontal counter, zero at end of
        H_FRONT mode
        h_counter <= (h_counter==H_FRONT)?10'd_0:(h_counter
            + 10'd_1) ;
        // Set hsync
        hsync_reg <= HIGH ;
        // State transition
        h_state <= (h_counter == H_FRONT)?H_PULSE_STATE:
            H_FRONT_STATE ;
    end
    if (h_state == H_PULSE_STATE) begin
        // Iterate horizontal counter, zero at end of
        H_FRONT mode
        h_counter <= (h_counter==H_PULSE)?10'd_0:(h_counter
            + 10'd_1) ;
        // Set hsync
        hsync_reg <= LOW ;
        // State transition
        h_state <= (h_counter == H_PULSE)?H_BACK_STATE:
            H_PULSE_STATE ;
    end
    if (h_state == H_BACK_STATE) begin
        // Iterate horizontal counter, zero at end of
```



```

926         H_FRONT mode
927         h_counter <= (h_counter==H_BACK)?10'd_0:(h_counter +
928             10'd_1) ;
929         // Set hsync
930         hsync_reg <= HIGH ;
931         // State transition
932         h_state <= (h_counter == H_BACK)?H_ACTIVE_STATE:
933             H_BACK_STATE ;
934         // Signal line complete at state transition (offset
935             by 1 for synchronous state transition)
936         line_done <= (h_counter == (H_BACK-1))?HIGH:LOW ;
937     end
938     //
939     //////////////////////////////////////
940
941     ////////////////////////////////////// VERTICAL
942     //////////////////////////////////////
943     //
944     //////////////////////////////////////
945
946     if (v_state == V_ACTIVE_STATE) begin
947         // increment vertical counter at end of line, zero
948             on state transition
949         v_counter <= (line_done==HIGH)?((v_counter==V_ACTIVE
950             )?10'd_0:(v_counter + 10'd_1)):v_counter ;
951         // set vsync in active mode
952         vsync_reg <= HIGH ;
953         // state transition - only on end of lines
954         v_state <= (line_done==HIGH)?((v_counter==V_ACTIVE)?
955             V_FRONT_STATE:V_ACTIVE_STATE):V_ACTIVE_STATE ;
956     end
957     if (v_state == V_FRONT_STATE) begin
958         // increment vertical counter at end of line, zero
959             on state transition
960         v_counter <= (line_done==HIGH)?((v_counter==V_FRONT)
961             ?10'd_0:(v_counter + 10'd_1)):v_counter ;
962         // set vsync in front porch
963         vsync_reg <= HIGH ;
964         // state transition
965         v_state <= (line_done==HIGH)?((v_counter==V_FRONT)?
966             V_PULSE_STATE:V_FRONT_STATE):V_FRONT_STATE ;
967     end
968     if (v_state == V_PULSE_STATE) begin
969         // increment vertical counter at end of line, zero
970             on state transition
971         v_counter <= (line_done==HIGH)?((v_counter==V_PULSE)
972             ?10'd_0:(v_counter + 10'd_1)):v_counter ;

```

```

956      // clear vsync in pulse
957      vsync_reg <= LOW ;
958      // state transition
959      v_state <= (line_done==HIGH)?((v_counter==V_PULSE)?
          V_BACK_STATE:V_PULSE_STATE):V_PULSE_STATE ;
960  end
961  if (v_state == V_BACK_STATE) begin
962      // increment vertical counter at end of line, zero
          on state transition
963      v_counter <= (line_done==HIGH)?((v_counter==V_BACK)
          ?10'd_0:(v_counter + 10'd_1)):v_counter ;
964      // set vsync in back porch
965      vsync_reg <= HIGH ;
966      // state transition
967      v_state <= (line_done==HIGH)?((v_counter==V_BACK)?
          V_ACTIVE_STATE:V_BACK_STATE):V_BACK_STATE ;
968  end
969
970  //
          //////////////////////////////////////
971  ////////////////////////////////////// COLOR OUT
          //////////////////////////////////////
972  //
          //////////////////////////////////////

973      red_reg      <= (h_state==H_ACTIVE_STATE)?((v_state==
          V_ACTIVE_STATE)?{color_in[7:5],5'd_0}:8'd_0):8'd_0 ;
974      green_reg    <= (h_state==H_ACTIVE_STATE)?((v_state==
          V_ACTIVE_STATE)?{color_in[4:2],5'd_0}:8'd_0):8'd_0 ;
975      blue_reg     <= (h_state==H_ACTIVE_STATE)?((v_state==
          V_ACTIVE_STATE)?{color_in[1:0],6'd_0}:8'd_0):8'd_0 ;
976
977  end
978  end
979  // Assign output values
980  assign hsync = hsync_reg ;
981  assign vsync = vsync_reg ;
982  assign red = red_reg ;
983  assign green = green_reg ;
984  assign blue = blue_reg ;
985  assign clk = clock ;
986  assign sync = 1'b_0 ;
987  assign blank = hsync_reg & vsync_reg ;
988  // The x/y coordinates that should be available on the NEXT
          cycle
989  assign next_x = (h_state==H_ACTIVE_STATE)?h_counter:10'd_0 ;

```

```

990     assign next_y = (v_state==V_ACTIVE_STATE)?v_counter:10'd_0 ;
991
992 endmodule
993
994
995
996
997 //=====
998 // M10K module for testing
999 //=====
1000 // See example 12-16 in
1001 // http://people.ece.cornell.edu/land/courses/ece5760/DE1\_SOC/
1002 // HDL\_style\_qts\_qii51007.pdf
1003 //=====
1004 module M10K_1000_8#(
1005     parameter integer DEPTH = 307200
1006 )(
1007     output reg [7:0] q,
1008     input [7:0] d,
1009     input [18:0] write_address, read_address,
1010     input we, clk
1011 );
1012     // force M10K ram style
1013     // 307200 words of 8 bits
1014     reg [7:0] mem [ DEPTH : 0 ] /* synthesis ramstyle = "
1015                                no_rw_check, M10K" */;
1016
1017     always @ (posedge clk) begin
1018         if (we) begin
1019             mem[write_address] <= d;
1020         end
1021         q <= mem[read_address]; // q doesn't get d in this clock
1022                                 cycle
1023     end
1024 endmodule

```

Listing 3: DE1_SoC_Computer.v